
Quner: Modular AI Framework for Intelligent Healthcare Training Systems

Project Report for MAJOR PROJECT II

submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

by

Name	Enrollment Number
Yash Tamar	R2142221249
Milind Vishwakarma	R2142221227

Under the supervision of

Mr. Mohd Farooq
Assistant Professor
Department of Computer Science



FEBRUARY – MAY 2026
SCHOOL OF COMPUTER SCIENCE
UNIVERSITY OF PETROLEUM AND ENERGY STUDIES
Dehradun-248007

CANDIDATE’S DECLARATION

We hereby certify that the project work entitled “**Quner: Modular AI Framework for Intelligent Healthcare Training Systems**” in partial fulfillment of the requirements for the award of the Degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING** with specialization in **Data Science, Big Data and Artificial Intelligence and Machine Learning** respectively, submitted to the School of Computer Science, UPES, Dehradun, is an authentic record of our work carried out during a period from **February, 2026** to **May, 2026** under the supervision of **Mr. Mohd Farooq, Assistant Professor, Department of Computer Science**.

The matter presented in this project has not been submitted by us for the award of any other degree of this or any other University.

Name	Enrollment Number
Yash Tamar	R2142221249
Milind Vishwakarma	R2142221227

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Mr. Mohd Farooq
Project Guide

Date: _____ 2026

Acknowledgement

We wish to express our deep gratitude to our guide **Mr. Mohd Farooq**, for all the advice, encouragement and constant support he has given us throughout our project work. This work would not have been possible without his support and valuable suggestions. We sincerely thank our respected **Dr. Virender Kadyan**, Head, Department of **Data Science**, for his great support in doing our project on **Quner: Modular AI Framework for Intelligent Healthcare Training Systems**. We are also grateful to the Dean, SoCS UPES, for giving us the necessary facilities to carry out our project work successfully. We also thank our Course Coordinator, **Dr. Neeraj Chugh**, and our Activity Coordinator, **Dr. Khushboo Jain**, for providing timely support and information during the completion of this project. We would like to thank all our friends for their help and constructive criticism during our project work. Finally, we express our sincere gratitude to our parents who have shown us this world and for every support they have given us.

Name	Enrollment Number
Yash Tamar	R2142221249
Milind Vishwakarma	R2142221227

Contents

1	Abstract	1
2	Introduction	1
2.1	Problem Statement	2
2.2	Objective	2
2.3	Purpose of the Project	3
2.4	Project Scope	4
2.5	Motivation	5
2.6	Literature Review	6
2.6.1	LLM-Driven Simulated Patients	6
2.6.2	Multi-Agent Systems for Software Automation	6
2.6.3	Retrieval-Augmented Generation	6
2.6.4	Rubric-Based Evaluation of Clinical Communication	7
2.6.5	Research Gap Identified	7
2.7	Key Research Areas	8
2.7.1	Multi-Agent System Design	8
2.7.2	LLM-Driven Conversational Simulation	8
2.7.3	Pluggable LLM Backends	8
2.7.4	Retrieval-Augmented Grounding	8
2.7.5	Rubric-Based Communication Evaluation	8
2.7.6	Browser-Native Multimodal Interaction	8
2.7.7	Operational Security for AI Products	9
3	Project Description	10
3.1	Architecture Overview	10
3.1.1	High-Level Components	10
3.1.2	Data & Storage Layer	11
3.1.3	Messaging & Communication	11
3.1.4	Security & Configuration	11
3.1.5	Typical Simulation Flow	12
3.1.6	Deployment Topology	12
3.2	Advantages of this Architecture	12
3.3	Technical Concepts	13
3.3.1	Tech Stack	13
3.3.2	User Interaction Layer	14
3.3.3	Backend Architecture	14
3.3.4	Deployment and Scalability	14
3.4	UML Diagrams	15
3.4.1	Use Case Diagram	17
3.4.2	Data Flow Diagram	19
3.4.3	Class Diagram	20
3.4.4	Sequence Diagram	21
3.4.5	Component Diagram	21
3.4.6	Architecture Diagram	23
3.5	SWOT Analysis	24
3.6	Methodology (Agile Model – Iterative Refactor)	25
3.6.1	Requirement Analysis	25
3.6.2	System Design	25
3.6.3	Implementation	26
3.6.4	Testing & Evaluation	26
3.6.5	Deployment & Scalability	26
3.6.6	Project Plan	27

4	Implementation	29
4.1	Project Structure	29
4.1.1	Root Level Project Overview	29
4.1.2	Detailed Directory Description	30
4.2	Code Explanation	32
4.2.1	Pipeline-Mode Dispatcher (<code>llm/clients.py</code>)	32
4.2.2	Generator Agent – 8-Step Vignette Pipeline (<code>agents/generator.py</code>)	33
4.2.3	VSP Agent – Routing + Post-Processing (<code>agents/vsp.py</code>)	33
4.2.4	Critic Agent – Hint (<code>agents/critic.py</code>)	35
4.2.5	Critic Agent – Final Feedback Schemas	35
4.2.6	Browser Speech Bridge (<code>client_dashboard/main.py</code>)	36
4.2.7	Dashboard WebSocket Consumer (<code>client_dashboard/utils.py</code>)	37
5	New Features Added in Major Project II	39
5.1	Operational Hardening	39
5.2	Multimodal User Interface	39
5.3	Backend Capability	40
5.4	Codebase Quality	40
6	Results and Output	42
6.1	Dashboard Overview	42
6.2	Patient Generation in Progress	42
6.3	Live Chat with the Virtual Patient	43
6.4	Right-Slide Vignette Drawer	43
6.5	Hint Notification	44
6.6	Final Feedback Dialog	44
6.7	Generation Log Dialog	45
6.8	Persisted Patient Artifact	46
7	System Requirements	47
7.1	Hardware Requirements (Host / Server)	47
7.2	Software Requirements	47
7.2.1	Operating System (Host)	47
7.2.2	Runtime & Dependencies	47
7.2.3	Browser (for Dashboard Use)	47
7.2.4	External Service Dependencies	48

Quner

1 Abstract

This project presents **Quner**, a modular agentic-AI framework that automates the full clinical-skills training cycle for medical students. The system orchestrates three cooperating agents – a *Generator agent* that produces evidence-based-medicine grounded vignettes with configurable Big-Five personalities, a *Virtual Simulated Patient (VSP) agent* that holds persona-consistent clinical interviews, and a *Critic agent* that delivers MIRS-aligned communication feedback alongside structured clinical evaluation [18, 33]. Conversation flows through a single asynchronous WebSocket hub that decouples the agents from the user-facing dashboard, allowing the same backend to drive a browser chat client, a Furhat robotic head, or both [24]. A pluggable *pipeline mode* lets the same code route through OpenAI, Groq’s free tier, or the original Cerebras + Groq + OpenAI mix, while a Pinecone-backed RAG layer can be turned on for clinical grounding when desired [22]. The browser dashboard supports text chat, push-to-talk speech input via the Web Speech API with optional auto-submit, system-default text-to-speech for patient replies, an on-demand ”Hint” button that asks the Critic agent for a one-sentence next-step suggestion, and a slide-out vignette drawer for quick reference [37]. Compared to traditional standardized-patient programmes, Quner reduces the marginal cost of one practice session to fractions of a cent while still producing structured, rubric-aligned feedback aligned with the Master Interview Rating Scale (MIRS) [14].

2 Introduction

Medical education depends heavily on *simulated practice*: students need a safe space to develop clinical interviewing, history-taking, and reasoning skills before they meet real patients. Traditional simulation programmes rely on standardized-patient (SP) actors, dedicated faculty time, and physical simulation suites [25]. These resources are scarce, expensive, and difficult to scale: a typical Objective Structured Clinical Examination (OSCE) station occupies a faculty member, an SP actor, and a physical room for the entire window in which a single student is being assessed [2].

The recent generation of large language models (LLMs) creates a credible alternative: an LLM-driven *virtual simulated patient* can sustain a fluent multi-turn dialogue, take on a personality, and stay consistent with a clinical vignette for the duration of a session [33]. Existing LLM-driven SPs, however, suffer from three repeatable failure modes [24]: (a) *medical inaccuracy*, where the model fabricates symptoms or guidelines that aren’t in the case; (b) *persona drift*, where the model breaks character or starts speaking like a doctor; and (c) *evaluative weakness*, where students get a transcript but no structured pedagogical feedback against a recognised rubric.

Quner addresses these failure modes with an explicit *multi-agent architecture*. Each of the three core capabilities – scenario generation, in-character dialogue, and standards-based feedback – is owned by a separate agent with its own prompt structure, model selection, and guardrails. The agents communicate through a single WebSocket message bus that also exposes the conversation to one or more user-facing clients. This separation lets each agent be tuned, evaluated, and replaced independently, and it makes the system robust to changes in any one upstream provider [9].

The project began with a research prototype that hard-coded a single set of API keys, was tightly coupled to a Furhat robotic head for I/O, and consisted of a small number of monolithic Python modules. Major Project II takes that prototype and turns it into a deployable, modular system: secrets are externalised, the

codebase is split into focused agent and LLM packages, and the dashboard is upgraded with browser-native speech, an on-demand coaching button, and a redesigned single-screen layout [26].

2.1 Problem Statement

Clinical interviewing is taught by *practice*, not by lectures. Yet practice in medical schools is bottlenecked by three tightly coupled scarce resources [25]:

- **Standardized patient actors** who must be recruited, trained, and paid for every cohort and every disease they need to portray.
- **Faculty observers** who score each encounter against the Master Interview Rating Scale (MIRS) or a similar communication rubric.
- **Physical OSCE rooms** which are limited in number and conflict with assessments and other clinical training.

The result is that students get a *small number of high-stakes practice sessions*, often clustered around exam season, with low formative feedback density [24]. Existing software simulators try to fill the gap, but the current generation of "AI patient" tools share a recurring set of weaknesses [2]:

- Vignettes are static, hand-authored, and shared between students, which limits replay value and exposes test items.
- Patient dialogue is driven by a single un-grounded LLM call, which produces fluent text but frequently hallucinates clinical content and breaks character when challenged [37].
- Feedback at the end of a session is, at best, a free-text summary – there is no per-criterion MIRS score, no evidence linkage to specific lines of the transcript, and no separate evaluation of clinical reasoning.
- The architecture is monolithic, which makes it hard to swap LLM providers in response to cost or rate-limit pressure, and it usually couples the system to a particular hardware setup (e.g., a robotic head).

There is therefore a need for a **unified, modular, AI-driven simulation framework** that combines *programmable, configurable scenario generation, persona-consistent and grounded patient dialogue, and structured, rubric-aligned, transcript-linked feedback*, while remaining cost-efficient and free of any single hardware or LLM-vendor dependency [18, 14].

2.2 Objective

The primary objective is to design and develop **Quner**, an autonomous, multi-agent simulation framework that lets medical students practise full clinical interviews with rich, structured feedback, on demand, at near-zero marginal cost [18]. Within that goal, the project pursues the following sub-objectives:

- **Programmatic Scenario Generation:** Build a *Generator agent* that, given a target difficulty (1–10) and a Big-Five personality vector, produces a complete clinical vignette including chief complaint, history of present illness, past medical history, medications, and personality-consistent communication style [33].

- **Persona-Consistent Patient Dialogue:** Implement a *Virtual Simulated Patient agent* with a routing pre-processor that classifies each doctor utterance into one of six conversational cases, selects the right system prompt, and post-processes the model output to keep the patient in character and within their plausible knowledge bounds [37].
- **Standards-Based Feedback:** Implement a *Critic agent* that, on session pause, generates per-criterion MIRS communication feedback (mark + explanation + transcript evidence) and a separate, structured clinical evaluation (diagnosis, treatment planning, follow-up, adherence to guidelines, risk assessment, test ordering, preventive care) [19].
- **On-Demand Coaching:** Add a *Hint* action that asks the Critic agent for a one-sentence next-step suggestion mid-conversation, without revealing the diagnosis [17].
- **Pluggable LLM Pipeline:** Drive every model call through a single `model_for(stage)` dispatcher that supports an OpenAI-only mode, a Groq-only mode (free tier), and the original full Cerebras + Groq + OpenAI pipeline, switchable by a single `PIPELINE_MODE` environment variable [38].
- **Optional RAG Grounding:** Allow a Pinecone + LlamaIndex retrieval layer to be enabled for clinical grounding when an evidence-based-medicine corpus is available [22].
- **Browser-Native Multimodal I/O:** Provide a NiceGUI browser dashboard with text chat, push-to-talk speech input via the Web Speech API, system-default text-to-speech for patient replies, an auto-submit toggle, a slide-out vignette drawer, and a hint button [25].
- **Modular, Maintainable Codebase:** Split the original monolithic `utils.py`, `prompts.py`, and `ui_utils.py` into focused `agents/`, `llm/`, `prompts/`, and `ui/` packages with thin facade modules that preserve backwards compatibility for any existing call sites [9].
- **Operational Safety:** Externalise all secrets to a `.env` file managed by `python-dotenv`, ensure no API key is ever hard-coded, and gate destructive features (RAG initialisation, full-pipeline cost) behind explicit configuration flags [31].
- **Furhat-Optional Design:** Decouple the dashboard from the Furhat robotic head so that text-chat works without any robot or ElevenLabs dependency, while preserving the option to wire Furhat back in for higher-fidelity setups.
- **Reproducible Practice:** Persist every generated vignette as a timestamped Markdown artifact in `generated_patients/` so that students and educators can re-run the same case for repeatable comparison [19].

2.3 Purpose of the Project

The primary purpose of this project is to create an *autonomous, multi-agent clinical-skills training platform* that lets a medical school multiply its effective practice-encounter throughput without scaling its standardized-patient or faculty headcount in proportion [25, 2]. The platform is designed to overcome the limitations of existing rule-based and single-prompt simulators by combining *programmatically generated, routed and grounded patient dialogue*, and *rubric-aligned evaluation* into a single asynchronous architecture [33, 18].

Quiner aims to ensure that medical students can practise interviewing whenever they want, against a configurable spread of disease difficulties and patient personalities, while educators retain visibility into the

per-criterion communication and clinical-reasoning scores attached to each session. By providing on-demand mid-conversation hints, students can self-coach during the encounter rather than only after it; by persisting each vignette to disk, an educator can replay the exact same case across an entire cohort for fair comparison [38, 14].

In addition to clinical training, the project serves as a working reference for *multi-agent system design with pluggable LLM backends*: every LLM call is routed through a single `model_for(stage)` dispatcher and a single `structured_response_format()` helper, both of which understand whether the deployment is running OpenAI-only, Groq-only, or the full Cerebras + Groq + OpenAI mix [9, 22]. The same architecture decouples the dashboard from any particular front-end stack: today the browser dashboard speaks to the WebSocket hub, but a Furhat robotic head, a CLI, or a future Slack bot can connect to the same hub by setting a single `role` field on the connection.

2.4 Project Scope

Quner is designed with operational efficiency, pedagogical fidelity, and a frictionless student experience at its core [25]. The scope of the project goes beyond a single chatbot: it integrates an explicit multi-agent backend, a NiceGUI-based browser dashboard, browser-native speech I/O, optional RAG grounding, and on-disk persistence for every generated case [33].

Educator & Program Director Scope

- Generate diverse vignettes across a configurable disease library at any difficulty level on demand [38].
- Replay the exact same vignette across an entire cohort using the persisted `generated_patients/<timestamp>_<dis` artifact.
- Receive per-criterion MIRS communication feedback and a separate structured clinical evaluation for each session.
- Inspect the full generation prompt-and-answer chain via the in-app generation log dialog [19].

Student / Trainee Scope

- Practise interviewing 24/7 with persona-consistent virtual patients.
- Use either typed text or push-to-talk speech input; receive patient replies as text and optionally as system-default text-to-speech [24].
- Request a one-sentence coaching hint mid-conversation without revealing the diagnosis [17].
- Pause the session at any time to receive structured MIRS + clinical feedback with line-level transcript evidence.
- Restart the same vignette with a single click to compare runs [14].

System Scope

- Generator, VSP, and Critic agents communicating asynchronously through a WebSocket hub on port 8085.

- Pluggable LLM dispatcher supporting OpenAI-only, Groq-only, and the full Cerebras + Groq + OpenAI pipeline [9].
- Optional Pinecone + LlamaIndex RAG layer initialised lazily only when `USE_RAG=1` [22].
- NiceGUI browser dashboard on port 8501 with a viewport-locked single-screen layout, slide-out vignette drawer, and browser-native speech I/O.
- Modular Python packages: `prompts/`, `llm/`, `agents/` on the backend and `ui/` on the dashboard, each with a thin facade module that preserves backwards compatibility [27].

Deployment and Infrastructure Scope

- All secrets (`OPENAI_API_KEY`, `GROQ_API_KEY`, `CEREBRAS_API_KEY`, `PINECONE_API_KEY`) live in a `.env` file loaded with `python-dotenv`; the `.env` is gitignored.
- Pipeline mode and RAG state are toggled via environment variables (`PIPELINE_MODE`, `USE_RAG`); no code change is required to switch backends.
- The two services – `ws_server.py` (backend) and `main.py` (dashboard) – are independent processes and can be containerised separately.

Application Scope

- Undergraduate and graduate medical curricula that need scalable interviewing practice.
- Continuing-education programmes that want repeatable assessment of communication competencies [25].
- Research into LLM-driven simulated patient design, since each agent and prompt is independently inspectable.

2.5 Motivation

- Medical schools are facing rising enrolment and falling per-student access to standardised-patient time; an on-demand AI patient meaningfully expands the practice surface area without requiring additional human SPs [25, 2].
- Existing single-prompt LLM patients hallucinate symptoms, break character under pressure, and give unstructured feedback. A multi-agent design with explicit routing, post-processing, and rubric-aligned evaluation directly addresses each of these failure modes [37, 24].
- LLM pricing and rate limits change rapidly. A pluggable pipeline lets the same code drop a paid OpenAI deployment in favour of Groq’s free tier (or vice-versa) without touching agent logic [18, 33].
- Browser-native Web Speech APIs (`SpeechRecognition`, `SpeechSynthesis`) make it possible to add multimodal I/O without external services, paid voice APIs, or custom hardware [19].
- Modular agent architectures are a known good design for problems where each capability has different reliability, latency, and cost profiles. Splitting Quner into Generator / VSP / Critic agents instead of one large prompt makes the system independently testable and replaceable [9, 27].

- Persisting every generated case as a Markdown artifact converts a one-shot generation into a reusable test item, which directly supports educator-led exam construction [26].
- Beyond medical education, the same architecture generalises to any role-play training problem – legal interviewing, customer-support coaching, behavioural therapy practice – making the framework reusable far beyond the immediate domain [14, 22].

2.6 Literature Review

The literature relevant to Quner spans four research areas: *LLM-driven simulated patients*, *multi-agent system design for software automation*, *retrieval-augmented generation for grounded reasoning*, and *rubric-based evaluation of clinical communication*. This section reviews the state of the art in each area and identifies the gap that Quner fills.

2.6.1 LLM-Driven Simulated Patients

Early simulated patients were rule-based: a finite-state machine produced canned responses keyed on doctor utterances [24]. The arrival of large language models opened a fluent, multi-turn alternative, but the first generation of LLM patients consisted of a single prompt that wrapped the vignette in a "you are a 45-year-old patient with hypertension" instruction [33]. Such systems are cheap to build and behave fluently for the first few turns, but they exhibit two recurring failures: *persona drift* (the model breaks character when asked clinical questions) and *knowledge leakage* (the patient cites diagnostic detail a real patient wouldn't know) [2].

Quner's VSP agent addresses both: each turn is routed through a pre-processing classifier that picks one of six conversation cases, each with its own system prompt; the raw model reply is then post-processed by a separate filter that strips out doctor-vocabulary, diagnostic hypotheses, and any content the patient could not plausibly know [37].

2.6.2 Multi-Agent Systems for Software Automation

The notion of decomposing complex automation tasks across cooperating agents has a long history in distributed AI [9]. The recent wave of "agentic AI" applies that pattern to LLM workloads: rather than asking a single model to do everything, separate agents own separate stages of a workflow and talk to each other through a shared bus [18, 14]. The advantages are well documented – independent prompt tuning, narrower context windows per call, and easier failure isolation [27] – but the design is harder to debug, prone to inter-agent loops, and requires careful state management.

Quner adopts the multi-agent pattern for clinical training: *Generator* owns scenario authoring, *VSP* owns dialogue, and *Critic* owns evaluation. Coordination flows through a single asynchronous WebSocket hub rather than a message broker, which keeps the implementation simple while still preserving the decoupling benefits [9].

2.6.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) couples a language model with a vector database so that factual answers are grounded in retrieved documents rather than the model's own parametric memory [22, 27]. In medical contexts, RAG is the standard mitigation against clinical hallucination [19].

Quner integrates RAG via Pinecone and LlamaIndex, gated behind `USE_RAG=1`: when on, the VSP agent's case-6 path queries the index for evidence-based-medicine context and injects it into the system

prompt; when off, the agent falls back to the default conversation system prompt and the heavy embedding stack is never imported [38].

2.6.4 Rubric-Based Evaluation of Clinical Communication

The Master Interview Rating Scale (MIRS) has been the dominant rubric for assessing physician interviewing in North American medical education for several decades. It scores a candidate on dozens of communication criteria from "Initiation of the encounter" to "Closing the encounter," typically on a 1–5 ordinal scale, and is paired in modern OSCE-style assessments with a separate clinical-reasoning rubric [19]. Software graders for free-text transcripts are an open research area; existing tools either score holistically (one score for the whole transcript) or focus on a single criterion such as empathy [17].

Quner’s Critic agent generates per-criterion MIRS scores along with explanations and direct transcript evidence, then separately evaluates clinical reasoning across seven criteria (diagnosis, treatment, follow-up, guideline adherence, risk assessment, testing, preventive care) [17]. The two evaluations are produced under structured-output JSON schemas (or via prompt-driven JSON when on Groq, where strict schemas aren’t supported), which keeps the grading deterministic and machine-readable.

2.6.5 Research Gap Identified

Area	Existing Tools	Limitation	Proposed System Advantage
Vignette Source	Hand-authored	Static, slow, leaks across cohorts	Programmatic generation by Generator agent at any difficulty + personality
Patient Dialogue	Single LLM prompt	Persona drift, knowledge leakage	Routed pre-processing + post-processing filter to keep persona and knowledge bounded
Feedback Style	Free-text or holistic score	No per-criterion rubric, no transcript evidence	Per-criterion MIRS + 7-criterion clinical evaluation with line-level evidence
LLM Backend	Vendor-locked	Cost / rate-limit fragility	Pluggable <code>model_for(stage)</code> dispatcher across OpenAI / Groq / full
I/O Modality	Text-only or robot-only	Either limited or hardware-dependent	Browser dashboard with text + Web-Speech STT/TTS + optional Furhat
Code Structure	Monolithic	Hard to extend	Modular <code>prompts/</code> , <code>llm/</code> , <code>agents/</code> , <code>ui/</code> packages with facade compatibility

Table 1: Research Gaps in Existing AI-Patient Literature

The combination of programmatic vignette generation, routed and post-processed persona-consistent dialogue, structured rubric-based feedback, pluggable LLM backends, and browser-native multimodal I/O

is, to our knowledge, not present in any single open-source clinical-training system. Quner contributes that combination as a working reference implementation [4, 32].

2.7 Key Research Areas

2.7.1 Multi-Agent System Design

Specialised agents communicating through a single bus is a well-studied pattern in distributed AI [9]. Modern agentic frameworks add chain-of-thought reasoning, persistent memory, and structured output, but they also surface new challenges: agent loops, hallucination, and coordination overhead [14]. Quner’s three-agent design is deliberately small to keep coordination tractable, and every cross-agent message passes through a single broadcast helper for auditability.

2.7.2 LLM-Driven Conversational Simulation

Generative models have transformed text-only chat applications, but role-play scenarios remain hard because they demand persistent persona, plausible knowledge bounds, and refusal of out-of-character requests [33]. Research highlights the value of *system-prompt routing* (different prompts for different conversation states) and *response post-processing* (a second model pass that checks the first one’s output) – both of which Quner adopts in the VSP agent [38].

2.7.3 Pluggable LLM Backends

The operational reality of LLM-based products is that pricing, latency, and rate limits change weekly. Decoupling agent logic from any single vendor is therefore an architectural priority rather than a “nice to have” [19]. Quner abstracts the choice through a single configuration variable and a small dispatcher, which lets the same code switch between OpenAI, Groq’s OpenAI-compatible endpoint, and the original full Cerebras + Groq + OpenAI mix without touching any agent or prompt [18].

2.7.4 Retrieval-Augmented Grounding

Vector-search-driven grounding is a standard mitigation for clinical hallucination but is also expensive and operationally heavy: it requires an embedding model, a vector index, and ingestion pipelines [22]. Quner’s design treats RAG as *optional*: the heavy LlamaIndex / HuggingFace / Pinecone stack is imported only when `USE_RAG=1`, so a deployment that doesn’t have an EBM corpus pays no boot cost [27].

2.7.5 Rubric-Based Communication Evaluation

Automated assessment of free-text clinical interviews is an active research problem [17]. Holistic scores are easy to produce but pedagogically weak; per-criterion scores with transcript evidence are pedagogically strong but harder to produce reliably. Quner generates both: a per-MIRS-criterion communication evaluation and a seven-criterion clinical-reasoning evaluation, each with explanations and transcript evidence [19].

2.7.6 Browser-Native Multimodal Interaction

Modern Chromium-based browsers expose SpeechRecognition and SpeechSynthesis APIs that handle on-device speech-to-text and text-to-speech without any server-side audio pipeline [4]. This dramatically

lowers the cost and complexity of multimodal training apps, but it also imposes browser-compatibility constraints. Quner uses these APIs directly with a graceful-degradation path: if STT isn't supported (e.g., Firefox) the user falls back to typing; if TTS is muted (the toggle is off) the patient still replies in text [32].

2.7.7 Operational Security for AI Products

Hard-coded API keys are an extremely common operational mistake in research code [31]. Quner externalises every secret to a `.env` file loaded by `python-dotenv`, gitignores the file, and provides a `.env.example` with sentinel placeholders – a small but important step in moving the system from "research demo" toward "deployable service" [6].

3 Project Description

3.1 Architecture Overview

Quner is structured as an asynchronous *three-agent multi-agent system* coordinated by a single WebSocket hub [9]. The Generator, VSP, and Critic agents own their own prompts, model selections, and post-processing logic, but never call each other directly: every event is broadcast through the hub, which lets the same backend simultaneously drive a NiceGUI browser dashboard, a Furhat robotic head, or any other client that speaks the WebSocket protocol [27]. A separate `model_for(stage)` dispatcher routes each LLM call to the right provider based on a single `PIPELINE_MODE` environment variable, decoupling agent logic from any specific vendor [18].

3.1.1 High-Level Components

- **Orchestrator (`ws_server.py`):** The asyncio WebSocket gateway. It accepts client connections, parses every JSON message into an action, dispatches the action to the right agent, and broadcasts replies back to all connected clients. Concretely it handles `generatePatient`, `launchPredefinedPatient`, `getPatientResponse`, `stopAndGetFinalFeedback`, `resumePatient`, `getHint`, and roughly a dozen status / history queries [31].
- **Generator Agent (`agents/generator.py`):** Owns scenario creation. Runs an eight-step generation pipeline: pick a disease from `conversion_tables.diseases`, reason about the requested difficulty, draft a vignette using the master template, scan it for inconsistencies, plan post-processing changes, regenerate the polished vignette, pick a Furhat face + ElevenLabs voice from the catalog, and start the resulting Patient via `globals.conversation_logic.start_patient()` [38]. Each step’s prompt and answer are appended to the debug log so the educator can audit the chain [19].
- **Virtual Simulated Patient Agent (`agents/vsp.py`):** Owns in-character dialogue. For every doctor turn (after the first), the agent runs a pre-processing call on the conversation history that classifies the situation into one of six cases – routine clinical question, non-clinical chitchat, deceptive utterance, etc. – and selects the matching system prompt. When case 6 fires and RAG is enabled, the agent queries the medical-knowledge tool for evidence-based context and injects it into the prompt. The raw answer is then post-processed by a separate filter that enforces personality, strips doctor-vocabulary, and removes information the patient couldn’t plausibly know [37].
- **Critic Agent (`agents/critic.py`):** Owns evaluation. `ask_quick_feedback()` runs after every doctor turn to produce a one-line actionable suggestion under the chat box. `final_feedback()` runs on session pause and emits both a structured clinical evaluation (seven criteria) and a per-criterion MIRS communication score (mark + explanation + transcript evidence). `generate_hint()` runs on demand when the doctor clicks the lightbulb button and emits a single-sentence next-step suggestion that is explicitly forbidden from naming the diagnosis [17].
- **LLM Service (`llm/clients.py`):** Boots the OpenAI / Cerebras / Groq clients based on `PIPELINE_MODE` and exposes a `model_for(stage)` dispatcher and a `structured_response.format(schema)` helper that picks `json_schema` for OpenAI and `json_object` for Groq [38].
- **RAG Service (`llm/rag.py`):** Lazily initialises Pinecone + HuggingFace embeddings + LlamaIndex when `USE_RAG=1`; otherwise exposes `medical_knowledge_tool = None` and the entire heavy stack is never imported [22].

- **Dashboard (`client_dashboard/main.py`):** NiceGUI-based browser application on port 8501. Connects to the WebSocket hub as a "dashboard" role client, renders a sticky header, the conversation scroll, the doctor input row, the immediate-feedback card, the scenario-settings card, and a slide-out vignette drawer. Includes a JavaScript module that wires the Web Speech API for push-to-talk STT and system-default TTS [25].

3.1.2 Data & Storage Layer

- **In-Memory Conversation State (`globals.conversation_logic.py`):** Holds the current Patient object, conversation history, debug log, quick-feedback string, accumulated final-feedback list, conversation ID, and patient-running flag. The module is server-wide global state – a known limitation of the original prototype that is preserved for backwards compatibility but documented as a refactor target.
- **Generated Patient Artifacts (`generated_patients/<ts>.<disease>.md`):** Every successful Generator run, and every predefined-patient launch, writes a Markdown file containing the source tag (generated or predefined_<n>), the conversation ID, the chosen face and voice, and the full vignette body. Educators can re-run the same case across a cohort by reusing the same artifact [19].
- **Pinecone Vector Index (optional):** When `USE_RAG=1`, an external Pinecone index named in `PINECONE_INDEX_NAME` stores the embeddings of the EBM corpus produced by `create_rag_storage.py`. The index is queried by the VSP agent's case-6 path [22].
- **Configuration (`.env`):** `OPENAI_API_KEY`, `GROQ_API_KEY`, `CEREBRAS_API_KEY`, `PINECONE_API_KEY`, `PIPELINE_MODE`, `USE_RAG`, `GENERATED_PATIENT_OUTPUT_DIR`, plus model-name overrides, all read by `config.py` via `python-dotenv`.

3.1.3 Messaging & Communication

- **WebSocket Hub:** The single point of inter-process communication. `ws_server.py` serves on `localhost:8085` and broadcasts every server-side event to every connected client via `globals.server.broadcast_message()` [27].
- **Dashboard → Backend actions:** `setRole`, `generatePatient`, `launchPredefinedPatient`, `getPatientResponse` (with or without `doctorResponse`), `stopAndGetFinalFeedback`, `resumePatient`, `getHint`, plus state-query actions (`checkPatient`, `getHistory`, `getQuickFeedback`, `recallFinalFeedback`, `getDebugLog`, `getPatientInformation`, `getConversationId`).
- **Backend → Dashboard events:** `generationStart`, `generationUpdate`, `generationStop`, `startPatient`, `stopPatient`, `patientGeneratedResponse`, `doctorResponse`, `quickFeedbackResponse`, `updateDebugLog`, `patientInformation` (now carrying fullVignette for the side-drawer), `feedbackResponse`, `clinicalFeedbackResponse`, `hintResponse`, `conversationId`.

3.1.4 Security & Configuration

- **Externalised Secrets:** Every API key lives in `.env`, loaded by `python-dotenv`; `.env` is gitignored. A `.env.example` is committed with sentinel values so a fresh clone has a clear template.

- **Pipeline Mode Switch:** `PIPELINE_MODE` defaults to `openai-only`. Setting it to `groq-only` routes every call through Groq’s free-tier OpenAI-compatible endpoint; setting it to `full` restores the original Cerebras-preprocessing + Groq-generation + OpenAI-feedback mix [18].
- **RAG Gating:** `USE_RAG=0` disables the heavy embedding stack entirely; the `LlamaIndex` / `HuggingFace` / `Pinecone` imports are skipped in `llm/rag.py`.

3.1.5 Typical Simulation Flow

1. **Trigger:** The student clicks *Generate patient* (or picks a predefined patient from the dropdown).
2. **Generation:** `ws_server.py` dispatches `generatePatient` to `agents.generator.generate_patient()`, which runs the eight-step pipeline. Each step broadcasts a `generationUpdate` event so the dashboard’s progress notification ticks forward.
3. **Patient Start:** The Generator instantiates a `Patient` object and calls `globals.conversation_logic.start_patient` which broadcasts `startPatient` and `patientInformation` (with the full vignette).
4. **Bootstrap:** The dashboard receives `startPatient new=true` and immediately sends a `getPatientResponse` (no `doctorResponse`) to ask the patient to greet the doctor first, using `prompts.conversation_new.firstmessage`.
5. **Conversation Loop:** The student types (or speaks) a doctor utterance. The dashboard sends `getPatientResponse {doctorResponse}`. The VSP agent runs preprocessing, picks a case, generates the answer, post-processes it, and broadcasts `patientGeneratedResponse`. The Critic agent then runs `ask_quick_feedback()` to update the immediate-feedback card.
6. **Optional Hint:** At any time the student can click the lightbulb. `getHint` fires; the Critic agent emits a `hintResponse` which the dashboard displays as a top notification.
7. **Pause:** The student clicks *Pause*. `stopAndGetFinalFeedback` is sent; the Critic agent runs `final_feedback()`, broadcasting one `clinicalFeedbackResponse` (seven clinical criteria) followed by twenty-five `feedbackResponse` events (one per MIRS criterion). The dashboard renders both in the feedback dialog.
8. **Resume / Restart:** The student can resume the same patient, or click *Restart (same sliders)* to re-run the Generator with the previous form values, or pick a different predefined patient.

3.1.6 Deployment Topology

- Two independent Python processes: `ws_server.py` on `:8085` and `main.py` (NiceGUI) on `:8501`.
- Both processes read the same project-root `.env`.
- Designed for Dockerisation; the smallest viable image installs only `openai`, `mdextractor`, `websockets`, `python-dotenv`, `nicegui<2`, and `aiohttp`. The RAG stack (`LlamaIndex` + `Pinecone` + `HuggingFace` embeddings) is an optional dependency tier.

3.2 Advantages of this Architecture

- **Independent Agent Tuning** – Generator, VSP, and Critic each have their own prompt files (`prompts/generation.py`, `prompts/vsp.py`, `prompts/feedback.py`, `prompts/post_processing.py`, `prompts/hint.py`), so improving one agent never silently affects the other two [9].

- **Pluggable LLM Backends** – Switching from OpenAI to Groq’s free tier (or back) is a single `.env` edit; no code change is required [18].
- **Optional RAG** – Deployments that have no EBM corpus pay no boot cost; deployments that do can flip `USE_RAG=1` and the same VSP code starts grounding case-6 answers [22].
- **Multi-Client by Design** – Because the orchestrator broadcasts every event, the same backend can drive a browser dashboard, a Furhat robotic head, and any future client (CLI, Slack bot) without changes.
- **Structured, Transcript-Linked Feedback** – The Critic agent emits per-criterion JSON with explanations and direct transcript evidence, which is pedagogically richer than holistic free-text feedback [17].
- **Browser-Native Multimodal I/O** – Web Speech API STT and system-default TTS run entirely on-device; no extra service, key, or audio pipeline is required [4].
- **Reproducible Cases** – Every generation persists to `generated_patients/`, turning a one-shot creation into a reusable test item [19].
- **Backwards-Compatible Refactor** – The `utils.py`, `prompts.py`, and `ui_utils.py` facades preserve every public symbol from the original monolithic versions, so any existing call site continues to work without modification [27].
- **Versatile Application** – The same architecture generalises to any role-play training problem – legal interviewing, customer-support coaching, behavioural therapy practice – making the framework reusable far beyond medical education.

3.3 Technical Concepts

3.3.1 Tech Stack

- **Core Runtime:** Python 3.10+, with type hints and modular packaging (`prompts/`, `llm/`, `agents/`, `ui/`). Async I/O via `asyncio` for the WebSocket hub [33].
- **LLM Providers:** *OpenAI* (`gpt-4o-mini` for vignettes, dialogue, post-processing, quick + full feedback, hints), *Groq* (`llama-3.3-70b-versatile` for vignette and answer generation in full / `groq-only` modes), *Cerebras* (`llama-4-scout-17b-16e-instruct` for VSP preprocessing in full mode). The provider is abstracted by a single OpenAI-compatible `client` reference per role [18].
- **Optional RAG Stack:** Pinecone (vector index), LlamaIndex (retrieval orchestration), HuggingFace `sentence-transformers/all-MiniLM-L6-v2` (CPU-friendly embeddings, 384-dim) [22].
- **WebSocket Gateway:** `websockets` (asyncio server), `aiohttp` (dashboard client) [27].
- **Browser Dashboard:** `nicegui<2` on top of FastAPI + Quasar, plus a hand-written JavaScript module that wires the Web Speech API for STT and TTS [25].
- **Configuration & Secrets:** `.env` via `python-dotenv`; `.env.example` committed as a template; `.env` gitignored [31].
- **Output Formatting:** OpenAI’s strict `json_schema` response format for feedback when on OpenAI; `json_object` mode plus prompt-side schema instructions when on Groq.

3.3.2 User Interaction Layer

- **Browser Dashboard (Primary Interface):** Sticky header with the project name and avatar status; a viewport-locked single-screen layout with a flex-1 conversation scroll, doctor input row, and immediate feedback card on the left, and scenario-settings + view-vignette button on the right.
- **Right-Slide Vignette Drawer:** `ui.dialog().props("position=right")` that holds the full vignette in a 480-px-wide panel with its own scroll area, opened on demand without leaving the chat.
- **Speech I/O:** A microphone button that toggles browser-native push-to-talk STT; an "Auto-submit after speaking" switch that programmatically clicks Send when the recognition ends; a "Speak responses" toggle that gates system-default TTS for patient replies [19].
- **Hint Notification:** A lightbulb button next to Send sends `getHint`; the response appears as a top notification with a dismiss button [17].
- **Generation Log Dialog:** A button that opens a modal showing every prompt and answer from the eight-step generation pipeline, available immediately after the patient is generated and after each predefined launch.
- **Furhat (Optional):** If a Furhat client also connects to the WebSocket hub with `role=furhat`, the same backend will route patient utterances to it for ElevenLabs-driven speech and Furhat-driven facial animation. The dashboard works unchanged whether or not Furhat is connected.

3.3.3 Backend Architecture

- **Orchestrator + Agent Packages:** `ws_server.py` dispatches actions to `agents.generator`, `agents.vsp`, `agents.critic`, `agents.predefined`, and `agents.lifecycle`. Each agent module imports the LLM clients from `llm.clients` and the prompt templates from `prompts/` [9].
- **Single Source of Truth for Models:** `llm.clients.model_for(stage)` is the only place that knows which model to use for a given stage in a given pipeline mode. Adding a new pipeline mode means changing only that function [18].
- **Lazy RAG:** `llm.rag` sets `medical_knowledge_tool = None` when `USE_RAG=0` and skips the heavy imports entirely, so the boot path is sub-second in OpenAI-only mode.
- **Compatibility Facade:** `utils.py` re-exports every public name from `llm/` and `agents/` so that `legacy import utils; utils.generate_patient(...)` call sites continue to work after the refactor [27].

3.3.4 Deployment and Scalability

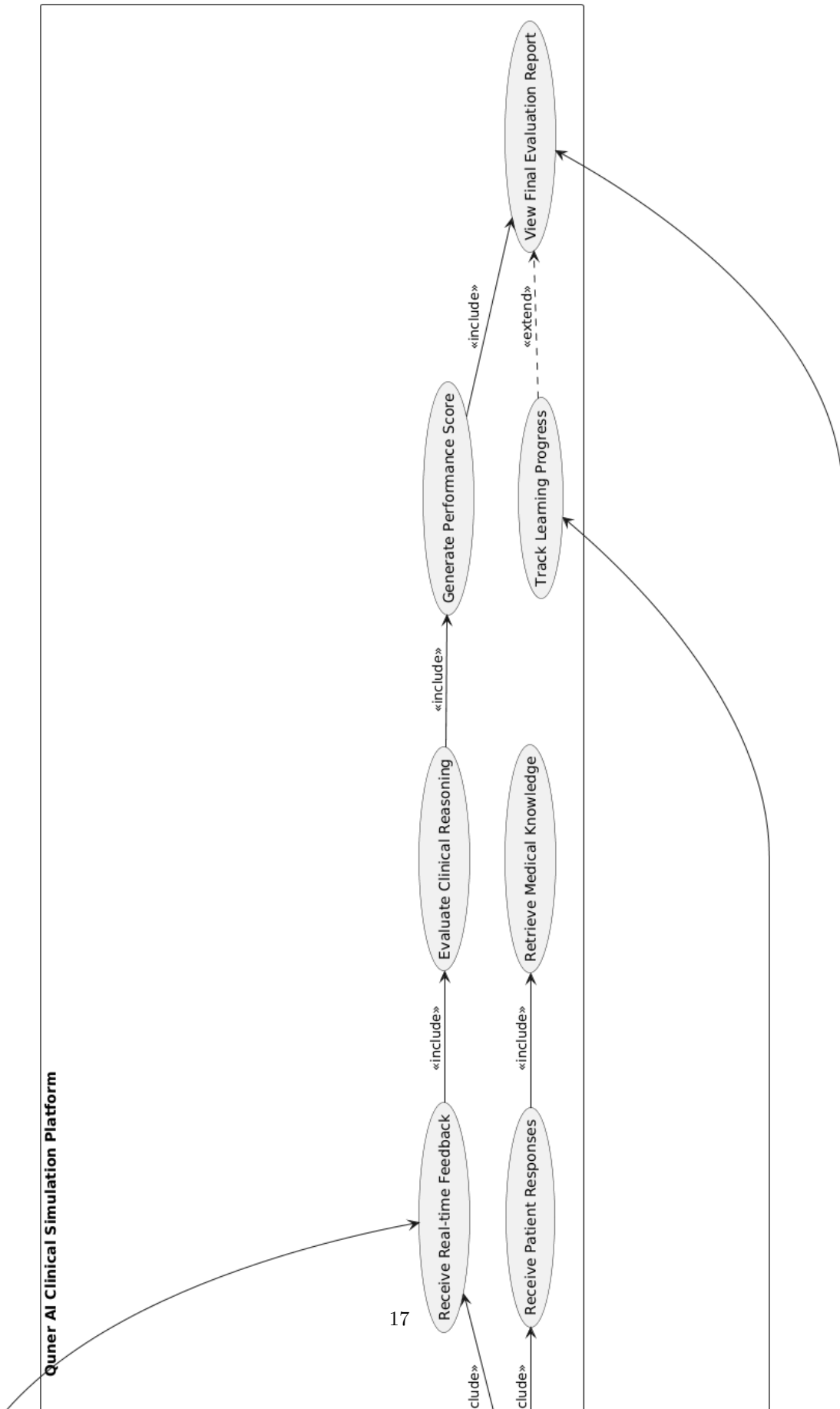
- **Two-Process Topology:** The backend and dashboard are independent processes that communicate only via WebSocket; either can be scaled or restarted without bringing the other down.
- **Slim Image:** The OpenAI-only deployment doesn't need the LlamaIndex / HuggingFace / Pinecone tier, which keeps the container image small and the cold-start fast [32].

- **Horizontal Scale:** The backend is stateful per-conversation but stateless across conversations; for multi-cohort use it can be horizontally scaled behind a sticky-session load balancer, with per-instance `generated_patients/` directories backed by shared object storage if persistence is needed [6].
- **Cost Controls:** `model_for(stage)` can be tuned per stage so that, e.g., the cheap preprocessing call uses a smaller / faster model while the expensive feedback call uses a stronger one.

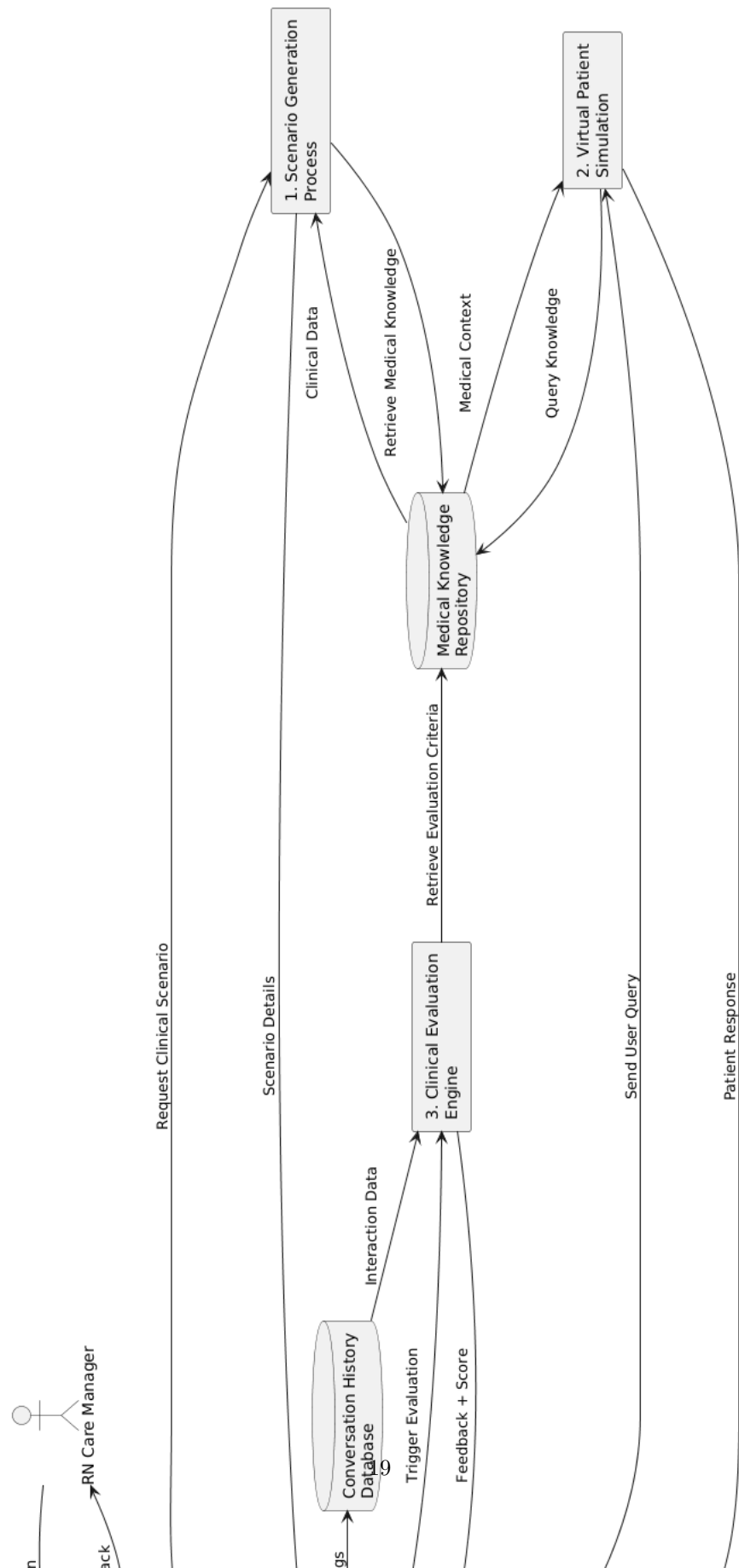
3.4 UML Diagrams

The following diagrams describe the static and dynamic structure of Quner. They were prepared during the SRS phase and are reproduced here unchanged.

3.4.1 Use Case Diagram



3.4.2 Data Flow Diagram



3.4.3 Class Diagram

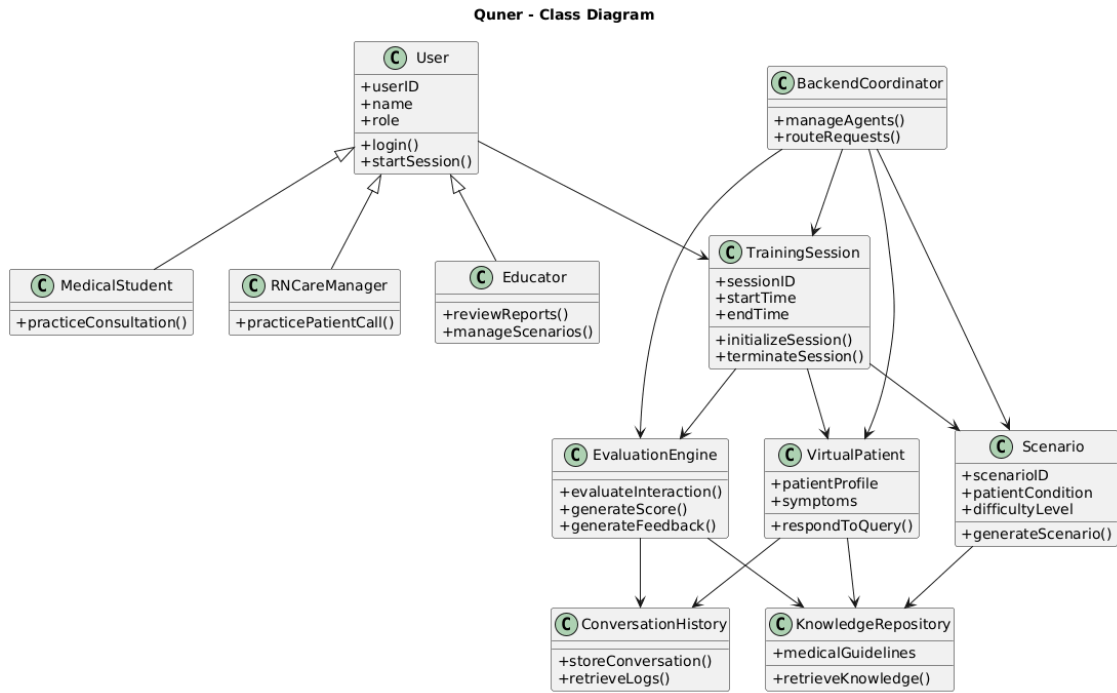


Figure 3: Class Diagram

3.4.4 Sequence Diagram

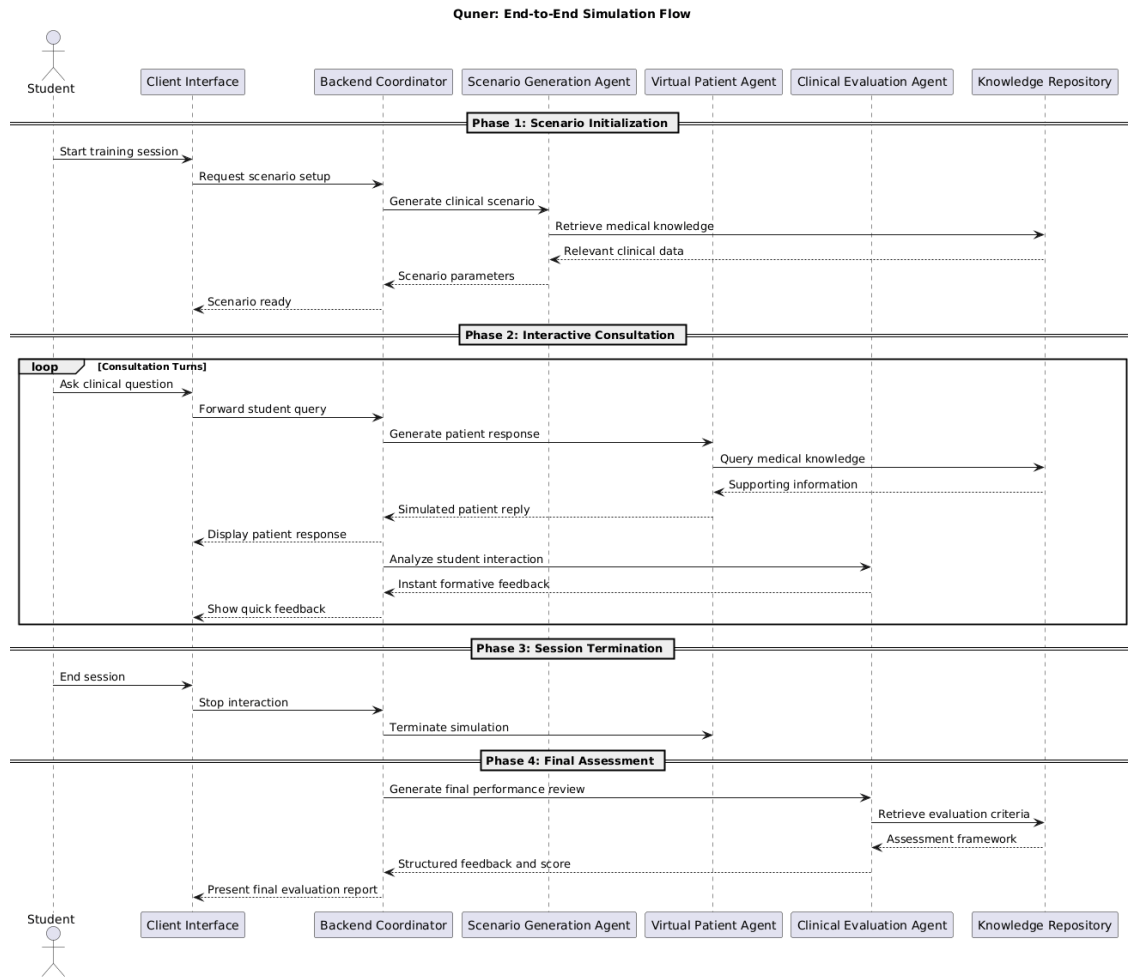


Figure 4: Sequence Diagram

3.4.5 Component Diagram

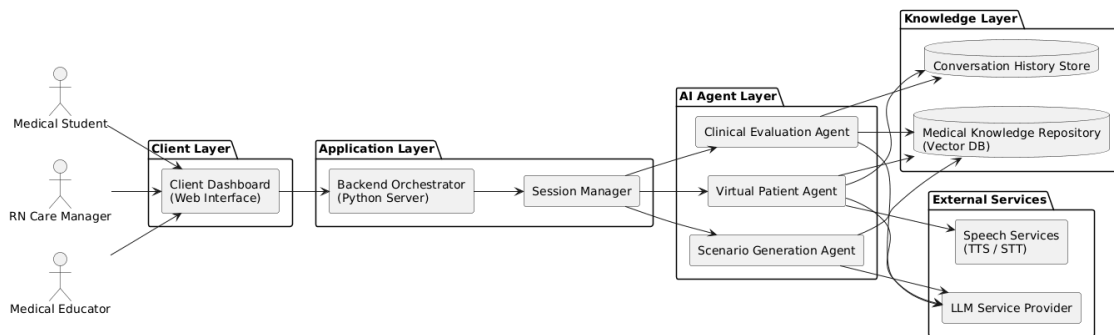
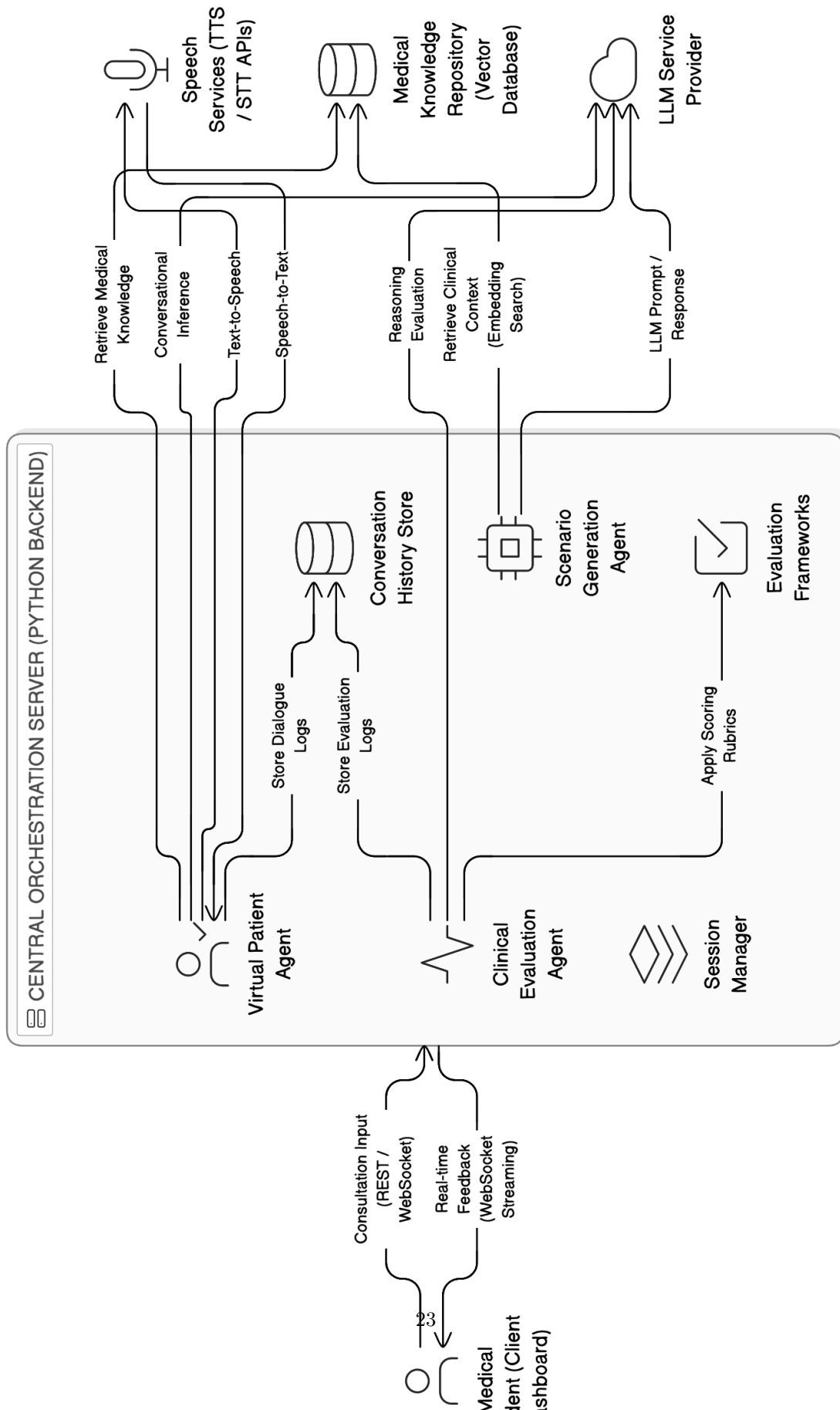


Figure 5: Component Diagram

3.4.6 Architecture Diagram



3.5 SWOT Analysis

Table 2: SWOT Analysis of Quner

Strengths	Weaknesses
Multi-Agent Decoupling: Generator, VSP, and Critic are independently tunable. Pluggable LLM Backends: OpenAI / Groq / full mode, switchable via one env var. Structured, Transcript-Linked Feedback: Per-criterion MIRS + 7-criterion clinical evaluation. Browser-Native Multimodal: STT and TTS via the Web Speech API, no extra services. Reproducible Cases: Every vignette persists to <code>generated_patients/</code>.	LLM Hallucination Risk: Without RAG enabled, factual grounding depends on the model alone. API Latency Sensitivity: Real-time chat is bounded by provider latency. No Physical Examination: The system simulates dialogue, not tactile clinical exams. Browser Compatibility: STT requires Chromium / Safari; Firefox falls back to typing.
Opportunities	Threats
Rising Demand for Scalable Training: Medical-school enrolment outpaces SP supply. Generalisable Architecture: The same design fits legal, customer-support, and therapy training. LMS Integration: MIRS / clinical scores can be exported into Moodle, Canvas, or Blackboard. Educator Test-Bank Construction: Persisted vignettes form a reusable test-item library.	LLM Pricing Volatility: Provider costs and rate limits change frequently. Educator Adoption Friction: Some faculty distrust AI-generated grading. Regulatory Scrutiny: Healthcare-adjacent software may attract privacy / accuracy review. Vendor Lock-In Pressure: Pluggable backends mitigate but do not eliminate this risk.

3.6 Methodology (Agile Model – Iterative Refactor)

The development of Quner followed an **Agile, iteration-driven methodology** closely aligned with Extreme Programming (XP) principles [25]. Because the project began with a research prototype and evolved through a series of incremental enhancements (multimodal I/O, pipeline-mode switching, modular package refactor, viewport-locked layout), an iterative process with short feedback cycles was the natural fit [33].

3.6.1 Requirement Analysis

Requirements were captured as *user stories* from the perspective of three stakeholder groups – educators, students, and operators – and refined collaboratively after each demo cycle [2].

- **User Stories (Functional):**

- As a *student*, I want to type or speak to a virtual patient so I can practise interviewing without a human SP.
- As a *student*, I want a one-line hint mid-conversation that doesn't reveal the diagnosis, so I can self-coach [17].
- As an *educator*, I want every session to produce a structured MIRS + clinical score with transcript evidence.
- As an *educator*, I want to replay the same vignette across a cohort for comparable assessment.
- As an *operator*, I want to switch LLM providers without touching agent logic when costs change.

- **Non-Functional:**

- Patient-reply latency target: under 5 s per turn on gpt-4o-mini.
- No hard-coded secrets, anywhere.
- Graceful degradation: STT failure or TTS failure must not crash the session; the chat must continue in text.
- Backwards compatibility: the refactor must preserve every public symbol from the original `utils.py`, `prompts.py`, `ui_utils.py` [27].

3.6.2 System Design

The system was redesigned around the principle of *single-responsibility modules with thin facades*.

- **Backend:** Split monolithic `utils.py` into `llm/clients.py`, `llm/rag.py`, `llm/parsers.py`, `agents/generator.py`, `agents/vsp.py`, `agents/critic.py`, `agents/predefined.py`, `agents/lifecycle.py`. `utils.py` survives as a 50-line facade that re-exports the public API [9].
- **Prompts:** Split monolithic `prompts.py` into `prompts/generation.py`, `prompts/vsp.py`, `prompts/feedback.py`, `prompts/post_processing.py`, `prompts/hint.py`, with a 90-line `prompts/__init__.py` that re-exports every template name.
- **Dashboard UI:** Split monolithic `ui_utils.py` into `ui/state.py` (all module-level mutable state), `ui/dialogs.py`, `ui/form.py`, `ui/chat.py`, `ui/sidebar.py`, `ui/feedback-view.py`, `ui/notifications.py`. A 60-line `ui_utils.py` facade preserves backwards compatibility.

- **Configuration Surface:** Every secret and pipeline-shaping flag is read from `.env`; `config.py` is the single source of truth for what those flags mean [31].
- **Workflow Example:** See the "Typical Simulation Flow" enumeration above for the trigger-to-feedback chain.

3.6.3 Implementation

Implementation followed XP practices: small commits, immediate live testing in the browser, refactor-as-you-go [32].

- **Agent Modules:** Generator pipeline (8 steps); VSP pre-processing → case routing → post-processing; Critic quick + final + hint.
- **LLM Integration:** `model_for(stage) dispatcher; structured_response_format(schema)` that returns `json-schema` for OpenAI and `json-object` for Groq.
- **Browser JavaScript Bridge:** `coworkSpeak()`, `coworkToggleMic()`, `coworkSetAutoSubmit()`, plus a wrapper-div trick to find the real `<input>` inside Quasar's `q-input` for STT injection.
- **XP Practices:** Iterative live demos; immediate refactor-on-discovery; user-driven scope adjustments (e.g., the voice-picker dropdown was added, then removed at the user's request).

3.6.4 Testing & Evaluation

Testing combined deterministic compile / import checks with live behavioural runs.

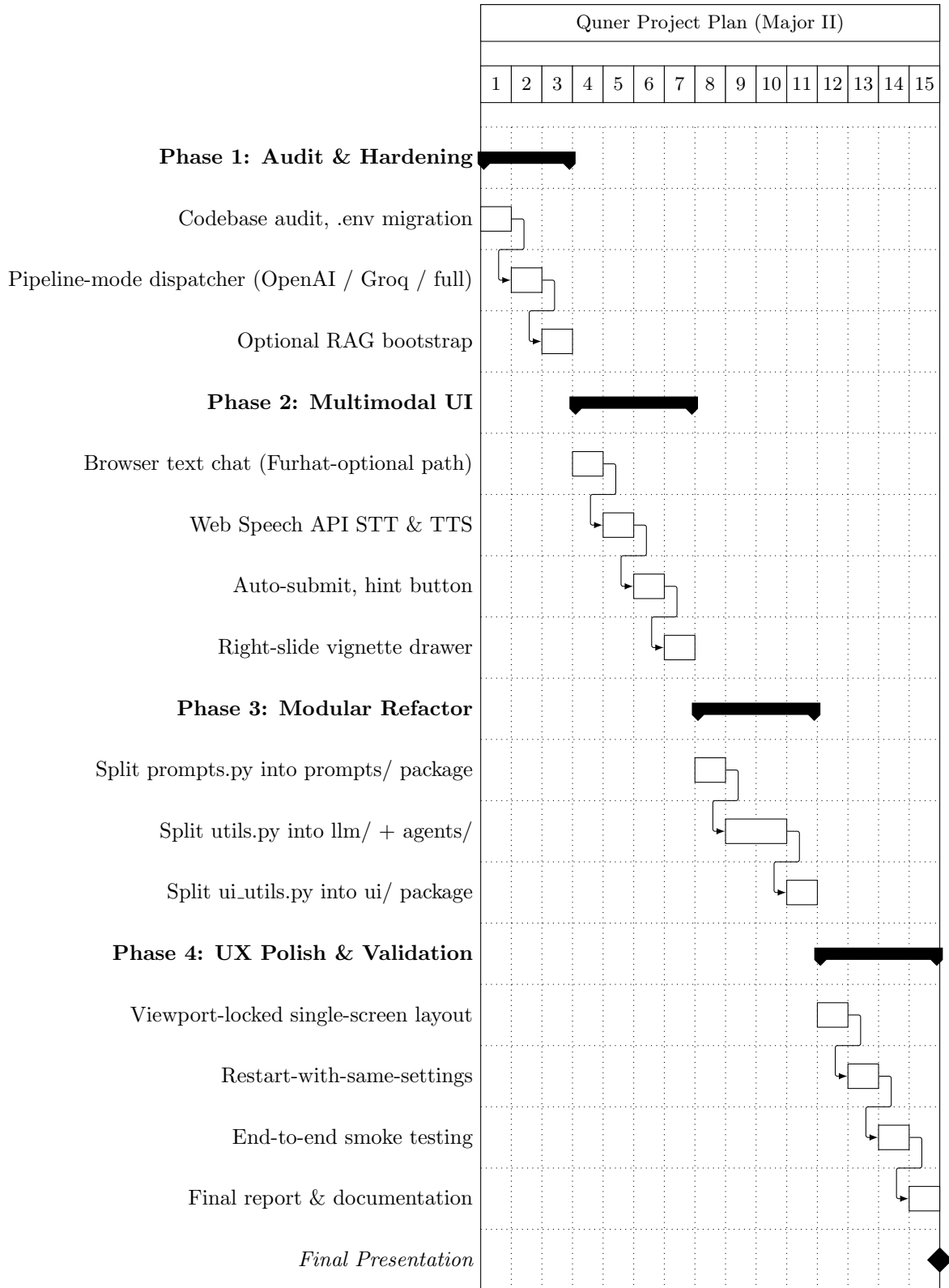
- **Static Checks:** Every change ran through `python -m py_compile` on the affected files.
- **Import-Chain Smoke Test:** A scripted import of `utils` (with stubbed `openai` / `mdextractor`) confirmed every public function survives the refactor.
- **Live Conversations:** Generated and predefined patients were exercised end-to-end through the dashboard.
- **Edge Cases:** Mic toggled mid-recognition (auto-submit suppressed); TTS toggled off mid-utterance (`speechSynthesis.cancel()` fires); rate-limit responses from Groq surfaced as visible error notifications.

3.6.5 Deployment & Scalability

- Two independent processes, both reading the same `.env`.
- Tested locally on macOS (Apple Silicon) with Python 3.13 and NiceGUI 1.4.x.
- The two processes are designed for separate Dockerisation and can be hosted on any Linux instance with outbound HTTPS to the chosen LLM provider.

3.6.6 Project Plan

The following Gantt chart represents the actual progression of Quner across the Major Project II window.



4 Implementation

4.1 Project Structure

4.1.1 Root Level Project Overview

Quner is a two-process Python application: a backend WebSocket hub (`ws_server.py`) that orchestrates three cooperating agents, and a NiceGUI browser dashboard (`main.py`) that connects to the hub as a “dashboard” client. The two processes share the same project-root `.env` and are otherwise independent [9].

The repository is organised so that each concern has its own focused package. The original prototype shipped one 844-line `utils.py`, one 879-line `prompts.py`, and one 653-line `ui_utils.py`. After the Major II refactor every source file is under 250 lines and grouped by purpose (LLM, agents, prompts, UI components, state) [27].

Frontend (NiceGUI Browser Dashboard) The dashboard is the primary student-facing surface. It is a NiceGUI application served on port 8501 that opens a sticky dark header, a viewport-locked split layout (chat + feedback on the left, scenario settings + view-vignette on the right), and a slide-out vignette drawer. A small embedded JavaScript module wires the Web Speech API for STT and TTS [25].

Backend (Python + Asyncio + LLM Providers) The backend hosts the agents and the WebSocket gateway. It uses `asyncio` for concurrency, the `openai` Python SDK as a single OpenAI-compatible client (pointed at OpenAI, Groq, or Cerebras depending on mode), and `websockets` for the gateway. Optional dependencies (Pinecone, LlamaIndex, HuggingFace embeddings) are imported lazily [33, 22].

4.1.2 Detailed Directory Description

```
quner/
|-- .env                                # Secrets + pipeline flags (gitignored)
|-- .env.example                        # Template
|-- README.md                          # Quick-start + architecture overview
|-- create_rag_storage.py              # Optional: build the Pinecone EBM index
|-- central_backend_server/            # Backend (port 8085)
|   |-- ws_server.py                  # Asyncio WebSocket gateway (orchestrator)
|   |-- config.py                     # .env-driven configuration
|   |-- conversion_tables.py           # Disease library, Big-Five, faces, voices
|   |-- predefined_patients.py         # Eleanor, Cassian, Marcus
|   |-- globals_server.py              # Connection registry + broadcast helper
|   |-- globals_conversation_logic.py  # Patient + conversation state
|   |-- utils.py                      # Backwards-compatible facade (50 lines)
|   |-- prompts/                      # ALL prompt templates
|       |-- __init__.py               # re-exports the flat namespace
|       |-- generation.py
|       |-- vsp.py
|       |-- feedback.py
|       |-- post_processing.py
|       |-- hint.py
|-- llm/                               # Client setup + RAG + parsers
|   |-- __init__.py
|   |-- clients.py                    # OpenAI / Groq / Cerebras + model_for
|   |-- rag.py                        # Lazy Pinecone + LlamaIndex
|   |-- parsers.py                    # markdown + face/voice JSON helpers
|-- agents/                           # The three agents + glue
|   |-- __init__.py
|   |-- generator.py                  # 8-step vignette pipeline
|   |-- vsp.py                        # Routing + post-processing dialogue
|   |-- critic.py                     # Quick + final feedback + hint
|   |-- predefined.py                 # Hard-coded patient launcher
|   |-- lifecycle.py                  # stop_patient + save_patient_artifact
|-- minimal_generate_client.py         # CLI test harness for Generator only
|-- generated_patients/                # Persisted vignettes (auto-created)
'-- client_dashboard/                  # Dashboard (port 8501)
    |-- main.py                       # NiceGUI entry point + injected JS
    |-- config.py                     # WebSocket URL + retry delay
    |-- utils.py                      # WS consumer + senders
    |-- ui_utils.py                   # Backwards-compatible facade (60 lines)
    '-- ui/                           # UI components
        |-- __init__.py
        |-- state.py                  # All module-level mutable state
        |-- dialogs.py                # Lazy modal dialogs
        |-- form.py                   # Sliders + Restart button
```

```

|-- chat.py          # chat_input + messages
|-- sidebar.py       # Controls + vignette drawer
|-- feedback_view.py # feedback() + per-criterion rows
`-- notifications.py # Progress + connection + show_hint

```

Most Important Backend Components:

- **agents/**: The three agents from the paper, each in its own file. Adding a new agent means dropping a file in this folder [9].
- **llm/clients.py**: The single source of truth for LLM provider selection and model dispatch [18].
- **llm/rag.py**: Lazy RAG bootstrap; never imports the heavy stack unless `USE_RAG=1` [22].
- **prompts/**: Every template grouped by agent for navigability.
- **ws_server.py**: 230-line action dispatcher that owns the WebSocket lifecycle [27].

Configuration (.env) Three pipeline modes are supported via a single env var:

- `PIPELINE_MODE=openai-only`: every LLM call goes through OpenAI, RAG is forced off, the `client` object is the OpenAI default.
- `PIPELINE_MODE=groq-only`: every LLM call goes through Groq’s OpenAI-compatible endpoint; structured-output is downgraded from `json-schema` to `json-object`; RAG forced off.
- `PIPELINE_MODE=full`: original paper pipeline – Cerebras for VSP preprocessing, Groq for vignette + answer generation, OpenAI for post-processing and feedback. Optional RAG via `USE_RAG=1`.

4.2 Code Explanation

The following snippets are excerpted from the refactored codebase. Each block is intentionally short – the full sources live in the indicated module.

4.2.1 Pipeline-Mode Dispatcher (llm/clients.py)

```
1  from openai import OpenAI
2  import config
3
4  print(f"Initializing language models (PIPELINE_MODE={config.PIPELINE_MODE})...")
5
6  if config.GROQ_ONLY:
7      client = OpenAI(base_url="https://api.groq.com/openai/v1/",
8                      api_key=config.GROQ_API_KEY)
9      cerebras_client = client
10     groq_client = client
11 elif config.OPENAI_ONLY:
12     client = OpenAI(api_key=config.OPENAI_API_KEY)
13     cerebras_client = client
14     groq_client = client
15 else:
16     client = OpenAI(api_key=config.OPENAI_API_KEY)
17     cerebras_client = OpenAI(base_url="https://api.cerebras.ai/v1/",
18                             api_key=config.CEREBRAS_API_KEY)
19     groq_client = OpenAI(base_url="https://api.groq.com/openai/v1/",
20                          api_key=config.GROQ_API_KEY)
21
22
23 def model_for(stage: str) -> str:
24     if config.GROQ_ONLY:
25         return config.vignette_generation_groq_model
26     if config.OPENAI_ONLY:
27         return {
28             "vignette": config.VSP_model_openai,
29             "preprocessing": config.VSP_model_openai,
30             "answergeneration": config.VSP_model_openai,
31             "postprocessing": config.VSP_model_postprocessing,
32             "quick_feedback": config.quick_feedback_api_model,
33             "full_feedback": config.full_feedback_api_model,
34         }[stage]
35     return {
36         "vignette": config.vignette_generation_groq_model,
37         "preprocessing": config.VSP_model_preprocessing,
38         "answergeneration": config.VSP_model_answergeneration,
39         "postprocessing": config.VSP_model_postprocessing,
40         "quick_feedback": config.quick_feedback_api_model,
41         "full_feedback": config.full_feedback_api_model,
42     }[stage]
```

The `llm/clients.py` module is the only place in the codebase that constructs LLM clients. The three branches above correspond to the three pipeline modes; the rest of the system never inspects `PIPELINE_MODE` directly. Switching from a paid OpenAI deployment to Groq’s free tier therefore costs one `.env` line; no agent is aware of the change [18].

4.2.2 Generator Agent – 8-Step Vignette Pipeline (`agents/generator.py`)

```

1  async def generate_patient(disease_difficulty, neuroticism, extraversion,
2                             openness, agreeableness, conscientiousness):
3      NR_OF_STEPS = 8
4      await broadcast_message({"action": "generationStart"})
5
6      # Step 1: pick a disease
7      disease_name = random.choice(list(conversion_tables.diseases.keys()))
8      disease_documents, score = conversion_tables.diseases[disease_name]
9      await globals_conversation_logic.add_debug_log(
10         f"1. Picked the disease: {disease_name} with a difficulty of {score}/10."
11     )
12
13     # Step 2-7: difficulty reasoning -> first draft -> consistency scan
14     #           -> post-process plan -> final draft -> avatar selection
15     # ... (full source in agents/generator.py)
16
17     patient = globals_conversation_logic.Patient(
18         disease=disease_name, vignette=markdown_block,
19         face=face, voice=voice,
20         neuroticism=conversion_tables.translate_score("neuroticism", neuroticism),
21         extraversion=conversion_tables.translate_score("extraversion", extraversion),
22         openness=conversion_tables.translate_score("openness", openness),
23         agreeableness=conversion_tables.translate_score("agreeableness",
24             agreeableness),
25         conscientiousness=conversion_tables.translate_score("conscientiousness",
26             conscientiousness),
27     )
28
29     await globals_conversation_logic.start_patient(patient)
30     save_patient_artifact(patient, source="generated")

```

The Generator agent runs an eight-step prompt chain on the configured *vignette* model. Each step appends both its prompt and its answer to the debug log (visible in the dashboard’s *Generation log* dialog), so the full provenance of every produced patient is auditable [19]. The final patient is persisted to `generated.patients/<timestamp>_<disease>.md` so the same case can be re-launched across an entire cohort [38].

4.2.3 VSP Agent – Routing + Post-Processing (`agents/vsp.py`)

```

1  async def generate_patient_response():
2      patient = globals_conversation_logic.get_patient()
3      history = globals_conversation_logic.get_conversation_history()
4
5      # Bootstrap path: first turn -> patient greets the doctor
6      if len(history) == 0:
7          system_prompt = prompts.conversation_new_firstmessage.substitute(...)
8      else:
9          # Pre-processing: classify the situation 1..6
10         completion = cerebras_client.chat.completions.create(
11             model=model_for("preprocessing"),
12             messages=[
13                 {"role": "system", "content": prompts.
14                  conversation_new_preprocessing},
15                 {"role": "user", "content": f"...history + vignette..."},
16             ],
17         )
18         output, database_question = _parse_output_and_question(
19             completion.choices[0].message.content)
20
21         match output:
22             case 1: system_prompt = prompts.conversation_new_1.substitute(...)
23             case 2: system_prompt = prompts.conversation_new_2.substitute(...)
24             case 3: system_prompt = prompts.conversation_new_3.substitute(...)
25             case 4: system_prompt = prompts.conversation_new_4.substitute(...)
26             case 5: system_prompt = prompts.conversation_new_5.substitute(...)
27             case 6:
28                 # RAG path (only if USE_RAG=1)
29                 if database_question and medical_knowledge_tool:
30                     rag = medical_knowledge_tool.query_engine.query(...)
31                     system_prompt = prompts.conversation_new_6.substitute(
32                         medical_information=rag.response, ...)
33
34         # Generate the raw answer
35         completion = groq_client.chat.completions.create(
36             model=model_for("answergeneration"), messages=llm_prompt)
37         response = completion.choices[0].message.content
38
39         # Post-process: enforce persona + knowledge bounds
40         response = post_process(was_doctor_utterance=True,
41                                answer=response,
42                                conversation_history=history,
43                                patient=patient)
44
45         await broadcast_message({"action": "patientGeneratedResponse",
46                                "response": response,
47                                "time": datetime.now().strftime("%H:%M")})

```

The VSP agent is the most interesting piece of agent logic in the system. Every doctor utterance after the first is run through a pre-processing classifier that picks one of six conversation cases. Case 6 is the only one that consults RAG; the others use case-specific system prompts that vary the tone and constraints. After the model produces the raw answer, a separate `post_process()` call runs on `gpt-4o-mini` with a personality-aware filter that strips doctor-vocabulary, removes diagnostic guesses, and tightens the wording to match the patient's Big-Five profile [37].

4.2.4 Critic Agent – Hint (`agents/critic.py`)

```

1  async def generate_hint():
2      patient = globals_conversation_logic.get_patient()
3      if patient is None:
4          await broadcast_message({"action": "hintResponse",
5                                  "hint": "Start a patient first."})
6          return
7
8      history = globals_conversation_logic.get_conversation_history()
9      convo = "".join(
10         f"{'Doctor' if e['origin']=='doctor' else 'Patient'}: {e['message']}\n"
11         for e in history
12     ) or "(no exchanges yet -- the patient has just walked in)"
13
14     prompt = prompts.hint_prompt.substitute(
15         vignette=patient.vignette,
16         conversation=convo,
17     )
18     try:
19         completion = client.chat.completions.create(
20             model=model_for("quick_feedback"),
21             messages=[{"role": "user", "content": prompt}],
22             temperature=0.4,
23         )
24         hint = completion.choices[0].message.content.strip()
25     except Exception as e:
26         hint = f"Hint unavailable right now ({type(e).__name__})."
27
28     await broadcast_message({"action": "hintResponse", "hint": hint})

```

The hint feature lets the student request a one-sentence next-step suggestion mid-conversation. The prompt is engineered to refuse to name the diagnosis, refuse to disclose vignette content the patient hasn't volunteered, and start its suggestion with an action verb (*Ask...*, *Explore...*, *Consider...*). This converts the Critic from a purely retrospective grader into an in-session coach [17].

4.2.5 Critic Agent – Final Feedback Schemas

```

1  clinical_schema = {

```

```

2     "type": "object",
3     "properties": {
4         "diagnosis_feedback": {"type": "string"},
5         "treatment_planning_feedback": {"type": "string"},
6         "follow_up_and_monitoring_feedback": {"type": "string"},
7         "adherence_to_guidelines_feedback": {"type": "string"},
8         "risk_assessment_feedback": {"type": "string"},
9         "test_and_investigation_ordering_feedback": {"type": "string"},
10        "preventive_care_feedback": {"type": "string"},
11    },
12    "required": [...],
13    "additionalProperties": False,
14 }
15 response_format = structured_response_format(clinical_schema)
16
17 per_criterion_format = structured_response_format({
18     "type": "object",
19     "properties": {
20         "mark": {"type": "number"},
21         "explanation": {"type": "string"},
22         "evidence": {"type": "array", "items": {"type": "string"}},
23     },
24     "required": ["mark", "explanation", "evidence"],
25     "additionalProperties": False,
26 })

```

The clinical schema produces a single JSON object covering seven evaluation criteria; the per-criterion schema runs once for every entry in `prompts.final_feedback_criteria` (the MIRS criteria list). On OpenAI both schemas are enforced through strict `json_schema`; on Groq they degrade to `json_object` mode plus prompt-side schema instructions, since Groq doesn't support the strict mode [19].

4.2.6 Browser Speech Bridge (`client_dashboard/main.py`)

```

1  window.coworkSpeak = function(text) {
2      if (!window.coworkTtsEnabled || !window.speechSynthesis) return;
3      try {
4          window.speechSynthesis.cancel();
5          const utter = new SpeechSynthesisUtterance(text);
6          utter.rate = 1.0; utter.pitch = 1.0;
7          window.speechSynthesis.speak(utter);
8      } catch (e) { console.warn('TTS failed:', e); }
9  };
10
11 function buildRecognition() {
12     const SR = window.SpeechRecognition || window.webkitSpeechRecognition;
13     if (!SR) { alert("Browser does not support speech recognition."); return null; }
14 }

```

```

14     const r = new SR();
15     r.continuous = false;           // push-to-talk
16     r.interimResults = true;
17     r.lang = 'en-US';
18     let finalTranscript = '';
19     r.onresult = (e) => {
20         let interim = '';
21         for (let i = e.resultIndex; i < e.results.length; i++) {
22             if (e.results[i].isFinal) finalTranscript += e.results[i][0].transcript
23                 + ' ';
24             else interim += e.results[i][0].transcript;
25         }
26         setInputValue((finalTranscript + interim).trim());
27     };
28     r.onend = () => {
29         // Auto-submit if enabled and not user-aborted
30         if (window.coworkAutoSubmit && !userManuallyStopped && hasText()) {
31             setTimeout(clickSend, 150);
32         }
33     };
34     return r;
35 }

```

The browser speech bridge runs entirely client-side. `coworkSpeak()` is gated on the "Speak responses" toggle and uses the system default voice. The recognition object is configured for push-to-talk (`continuous=false`); when it ends naturally and the auto-submit switch is on, a synthetic click on the Send button posts the recognized text to the backend [4, 32].

4.2.7 Dashboard WebSocket Consumer (`client_dashboard/utils.py`)

```

1  async def consumer():
2      global _websocket
3      while True:
4          try:
5              async with aiohttp.ClientSession(trust_env=True) as session:
6                  async with session.ws_connect(config.WS_CONN) as websocket:
7                      _websocket = websocket
8                      await websocket.send_str('{"action":"setRole","role":"dashboard"}')
9                      # Replay current state to repopulate UI on reconnect
10                     for action in ("checkPatient", "getHistory", "getQuickFeedback",
11                                     "recallFinalFeedback", "checkFurhatConnected",
12                                     "getDebugLog", "getPatientInformation",
13                                     "getConversationId"):
14                         await websocket.send_str(f'{{"action":"{action}"}}')
15                     async for message in websocket:

```

```
16             data = message.json()
17             # ... dispatch ~15 action types to ui_utils handlers ...
18         except Exception as e:
19             print(f"WS error: {e}. Retrying in {config.RETRY_DELAY}s.")
20         finally:
21             _websocket = None
22             await asyncio.sleep(config.RETRY_DELAY)
```

The dashboard's consumer task is a single async loop that maintains the WebSocket connection. On (re)connect it replays a fixed sequence of state-query actions so the UI repopulates from the backend's current state, then listens for events and dispatches them to the right `ui_utils` handler. Connection loss is handled by sleeping `RETRY_DELAY` seconds and reconnecting automatically [31].

5 New Features Added in Major Project II

This section enumerates the enhancements introduced during Major Project II on top of the original SRS-stage prototype. Every item below is fully implemented and is exercised in the screenshots in the next section.

5.1 Operational Hardening

- **Externalised Secrets via `.env`.** The original `config.py` contained four API keys hard-coded as Python string literals – a serious operational issue for any pushed commit. Major II moves all four (`OPENAI_API_KEY`, `GROQ_API_KEY`, `CEREBRAS_API_KEY`, `PINECONE_API_KEY`) plus the new pipeline-shaping flags into a project-root `.env`, loaded by `python-dotenv`. The `.env` is gitignored and a `.env.example` template is committed [31].
- **Pipeline-Mode Dispatcher.** A single `PIPELINE_MODE` variable selects between three modes – `openai-only`, `groq-only`, and `full` – and a single `model_for(stage)` function in `llm/clients.py` owns every model name decision. Switching providers therefore costs one env-var edit and one process restart.
- **Optional RAG.** The Pinecone + LlamaIndex + HuggingFace embedding stack is imported lazily inside `llm/rag.py` and only when `USE_RAG=1`. The OpenAI-only path therefore boots in well under a second and never touches the heavy dependencies [22].
- **Furhat-Optional Architecture.** `globals.conversation_logic.start_patient()` no longer treats "no Furhat" as a warning; it always broadcasts `startPatient`, allowing the dashboard to switch into chat mode regardless of robot presence. The same backend can still drive a real Furhat if one connects with `role=furhat`.

5.2 Multimodal User Interface

- **Browser Text Chat.** A NiceGUI input + Send button row replaces the original Furhat-only voice loop. The dashboard auto-bootstraps the patient's first turn by sending `getPatientResponse` as soon as `startPatient` arrives, so the conversation kicks off with a patient greeting instead of silence [25].
- **Push-to-Talk Speech-to-Text.** A microphone button next to Send toggles `webkitSpeechRecognition` (or `SpeechRecognition` where available). Interim and final transcripts stream into the chat input as the user speaks; an alert is raised if the browser doesn't support the API [4].
- **Auto-Submit After Speaking.** A toggle that, when on, programmatically clicks the Send button when speech recognition ends. Manual mic aborts (clicking the mic again) are tracked separately so they never trigger an auto-send [32].
- **Text-to-Speech for Patient Replies.** A "Speak responses" switch routes every `patientGeneratedResponse` through the browser's `SpeechSynthesis` API with the system default voice. The toggle gates a single `coworkSpeak()` JavaScript function called from `add_patient_message()`.
- **Hint Button (Mid-Session Coaching).** A lightbulb button next to Send sends a `getHint` action; the Critic agent replies with a one-sentence next-step suggestion that is forbidden from naming the diagnosis. The hint pops up as a top notification with a dismiss button [17].

- **Right-Slide Vignette Drawer.** A `ui.dialog().props("position=right")` 480-px-wide panel that holds the full vignette in a scrollable area, opened on demand from a "View patient vignette" button. The drawer replaces the previous modal-button experience and lets the student keep the case in view during the chat.
- **Restart with Same Settings.** The most recent launch is tracked (*generated* or *predefined* N); a Restart button re-runs the same generation parameters or relaunches the same predefined patient with one click.
- **Viewport-Locked Single-Screen Layout.** The page itself never scrolls; only the chat area does. The header is fixed; the conversation flexes to fill the available height; the immediate-feedback card is capped at 130 px with internal scroll. The right column has its own internal scroll for narrow viewports.
- **Polished Theme.** Sticky dark header with the project name and avatar status; emerald-green primary actions; Tailwind-style spacing; custom scrollbars; mic-active red-pulse styling.

5.3 Backend Capability

- **Hint Backend Path.** `generate_hint()` in `agents/critic.py` plus a `getHint` action handler in `ws_server.py` (with a graceful "no patient yet" path).
- **Full Vignette over WebSocket.** `patientInformation` now carries a `fullVignette` field so the side-drawer can render the complete vignette without an extra round-trip.
- **Synthetic Doctor Bootstrap.** The VSP agent injects a [*Doctor enters the room and is ready to begin.*] synthetic user turn for the first patient utterance, since OpenAI rejects system-only prompts.
- **EBM-Optional Final Feedback.** `final_feedback()` now tolerates missing EBM documents and falls back to "use general medical knowledge" instead of crashing on a `FileNotFoundError`.
- **JSON-Object Fallback for Groq.** `structured_response_format()` returns `json_schema` for OpenAI but `json_object` for Groq (which doesn't support strict schemas), and the prompts append a schema-shape instruction in Groq mode so the output stays parseable [19].

5.4 Codebase Quality

- **Modular Refactor of Backend.** The 844-line `utils.py` is now a 50-line facade re-exporting from `llm/clients.py`, `llm/rag.py`, `llm/parsers.py`, and the five files in `agents/`. Each split file is single-responsibility [9].
- **Modular Refactor of Prompts.** The 879-line `prompts.py` is now a `prompts/` package (5 submodules + an `__init__.py` facade), grouped by agent.
- **Modular Refactor of Dashboard UI.** The 653-line `ui_utils.py` is now a 60-line facade over a `ui/` package with seven focused submodules (`state`, `dialogs`, `form`, `chat`, `sidebar`, `feedback_view`, `notifications`).
- **NiceGUI 2.x Compatibility Layer.** Lazy dialog creation (`fresh_*_dialog()` now creates the underlying `ui.dialog()` only the first time it runs) plus pinning to `nicegui<2.0` so the dashboard works against either generation of the framework [27].

- **Backwards-Compatible Facades.** Every public symbol from the original `utils.py`, `prompts.py`, and `ui_utils.py` is preserved through facade modules, so legacy call sites continue to work without modification.

6 Results and Output

The following figures demonstrate Quner running end-to-end. All screenshots come from a live browser session against the OpenAI-only deployment with a freshly generated patient.

6.1 Dashboard Overview



Figure 7: Dashboard overview: viewport-locked single-screen layout.

Figure 7 shows the dashboard at first load, before any patient is selected. The sticky header carries the project name and the avatar status icon. The left column hosts the conversation log; the right column hosts the scenario-settings card and the *View patient vignette* button.

6.2 Patient Generation in Progress

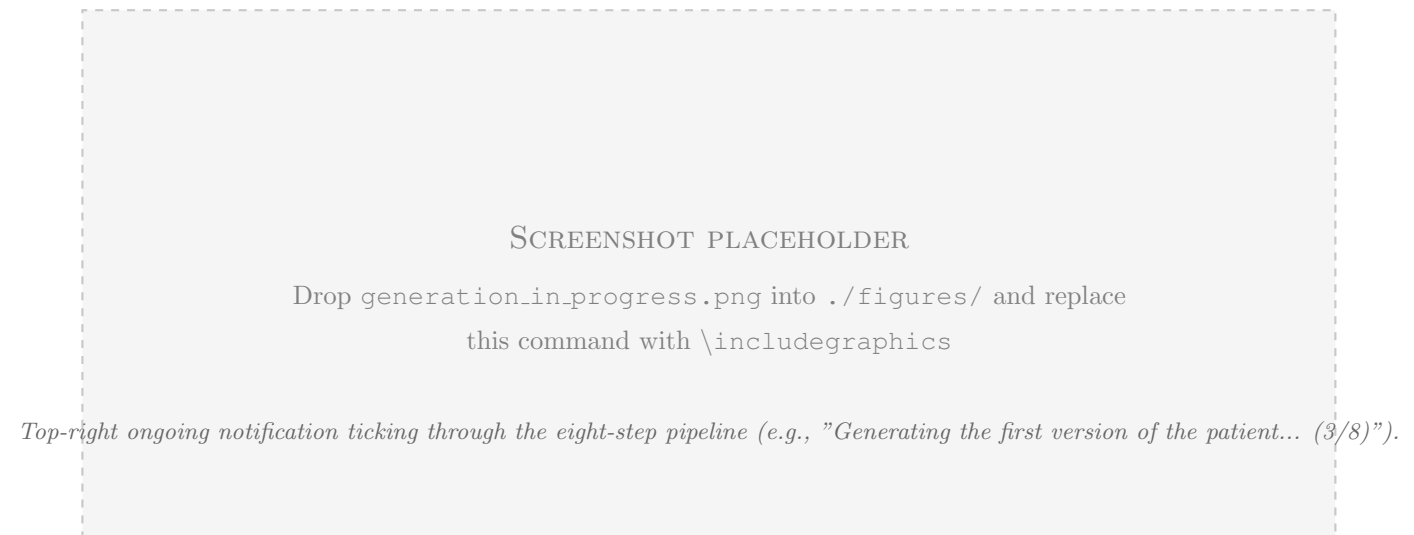


Figure 8: Eight-step generation pipeline progress notification.

Figure 8 captures the Generator agent in mid-flight. Each `generationUpdate` event from the backend updates the on-screen progress notification with the current step number and a human-readable description.

6.3 Live Chat with the Virtual Patient

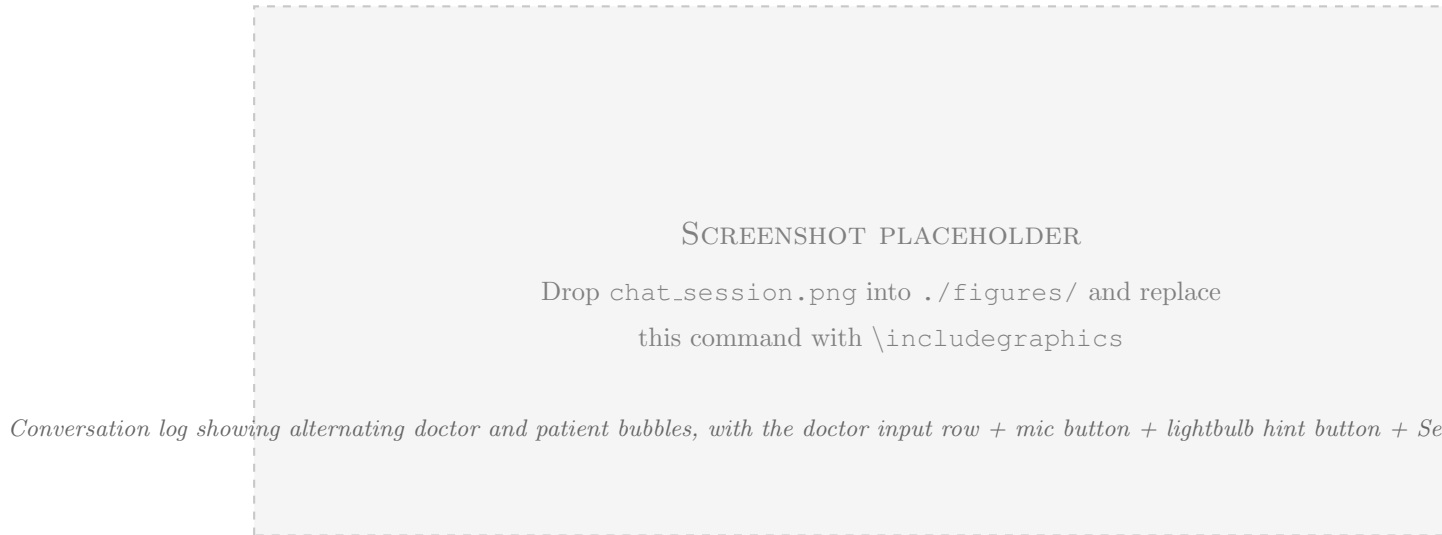


Figure 9: Live chat session: persona-consistent dialogue with text + speech I/O.

Figure 9 shows a live consultation. Doctor utterances appear right-aligned with the doctor avatar; patient replies appear left-aligned with the patient avatar. The skeleton placeholder beneath the most recent doctor utterance is visible while the VSP agent generates the next reply.

6.4 Right-Slide Vignette Drawer



Figure 10: Right-slide vignette drawer: full case file accessible mid-chat.

Figure 10 shows the slide-out vignette drawer overlaying the right side of the chat. Clicking the *View patient vignette* button slides this 480-px panel in from the right edge; closing it returns the layout to the previous viewport-locked state.

6.5 Hint Notification



Figure 11: Hint notification: on-demand mid-session coaching from the Critic agent.

Figure 11 shows the hint notification produced after the student clicks the lightbulb. The Critic agent obeys its prompt-level constraints: it never names the diagnosis, never reveals undisclosed vignette content, and starts the suggestion with an action verb.

6.6 Final Feedback Dialog

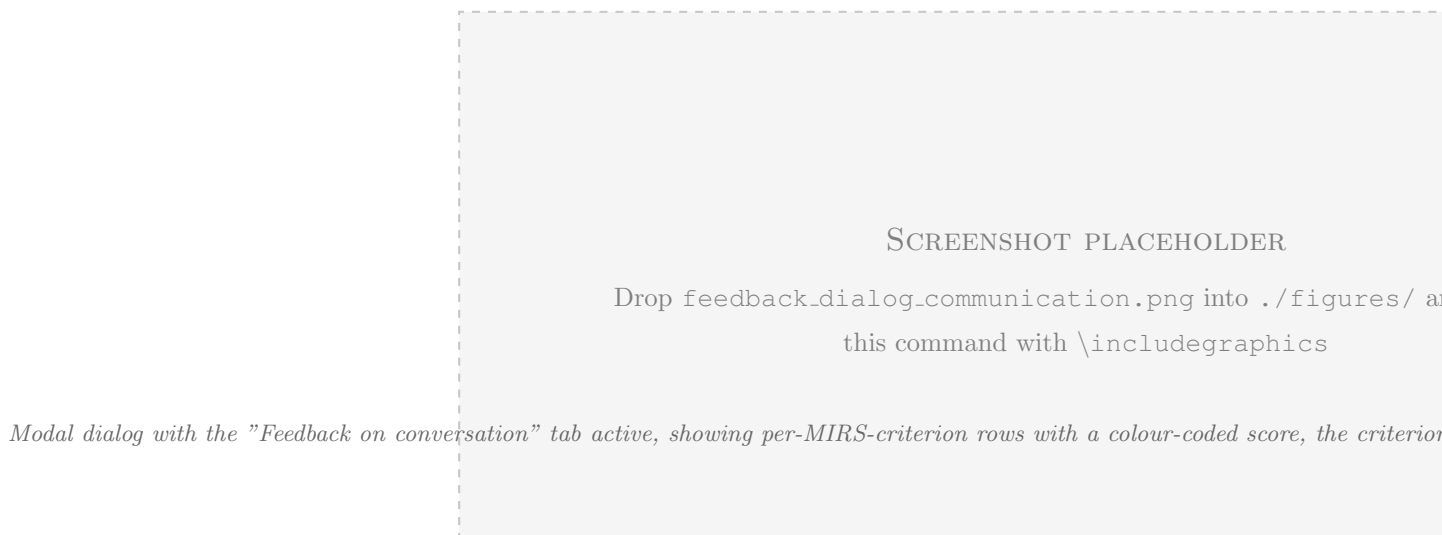


Figure 12: Final feedback (communication tab): per-criterion MIRS scoring with transcript evidence.

Figure 12 shows the communication-feedback tab. Every criterion is rendered as one row with a circle-coloured score (red = 1–2, yellow = 3, green = 4–5), the criterion name, the explanation, and quoted transcript lines that justify the score.

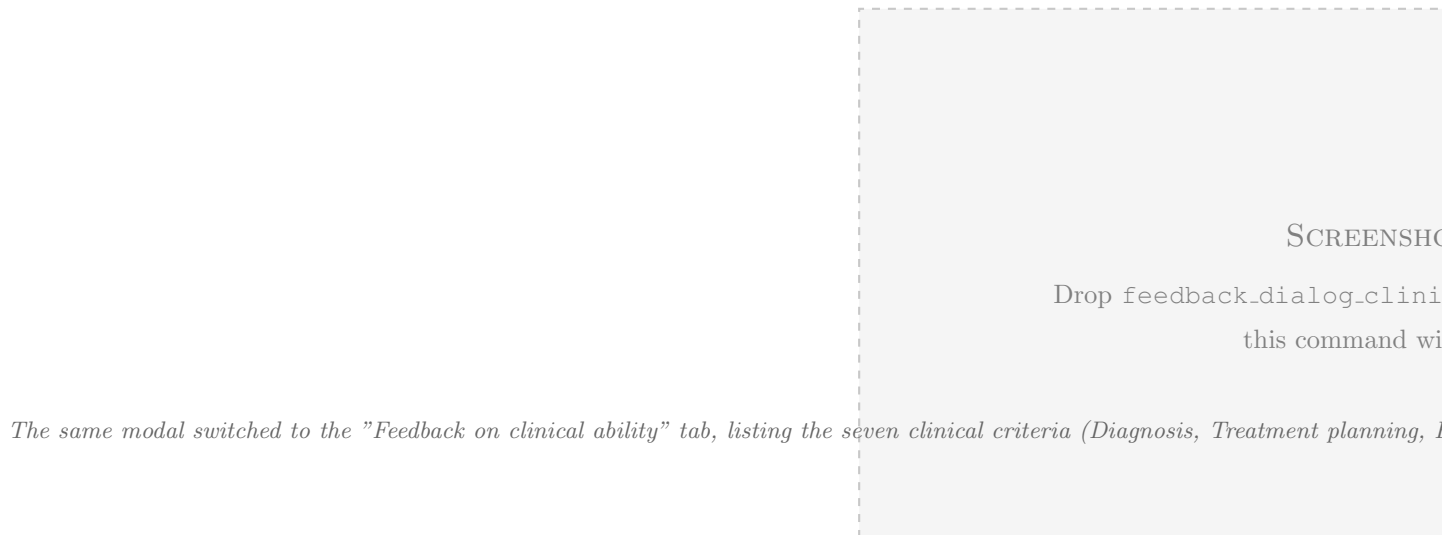


Figure 13: Final feedback (clinical tab): seven-criterion structured clinical evaluation.

Figure 13 shows the clinical-feedback tab of the same modal. The seven criteria are produced as a single LLM call with a strict JSON schema (or json_object plus prompt-side instructions on Groq).

6.7 Generation Log Dialog

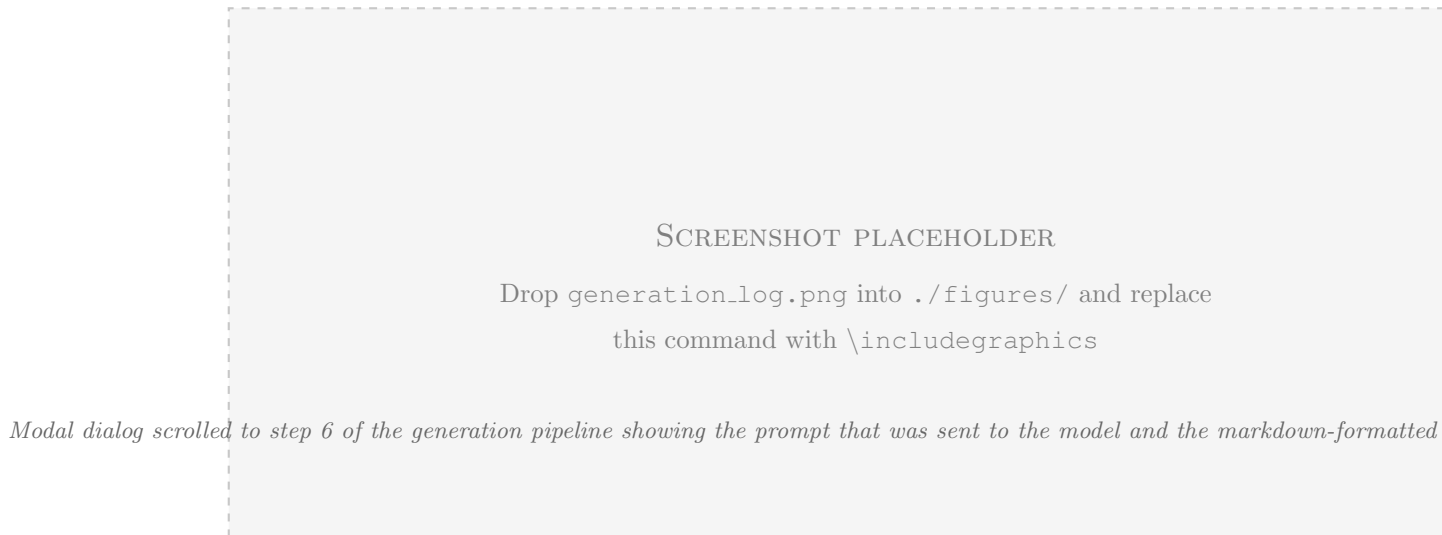


Figure 14: Generation log: full prompt + answer trail for the eight-step Generator pipeline.

Figure 14 shows the generation-log dialog. Every prompt and every answer in the eight-step Generator pipeline is appended to the log; educators can audit the full chain from disease pick to face/voice selection to confirm that the produced vignette matches their intended difficulty.

6.8 Persisted Patient Artifact

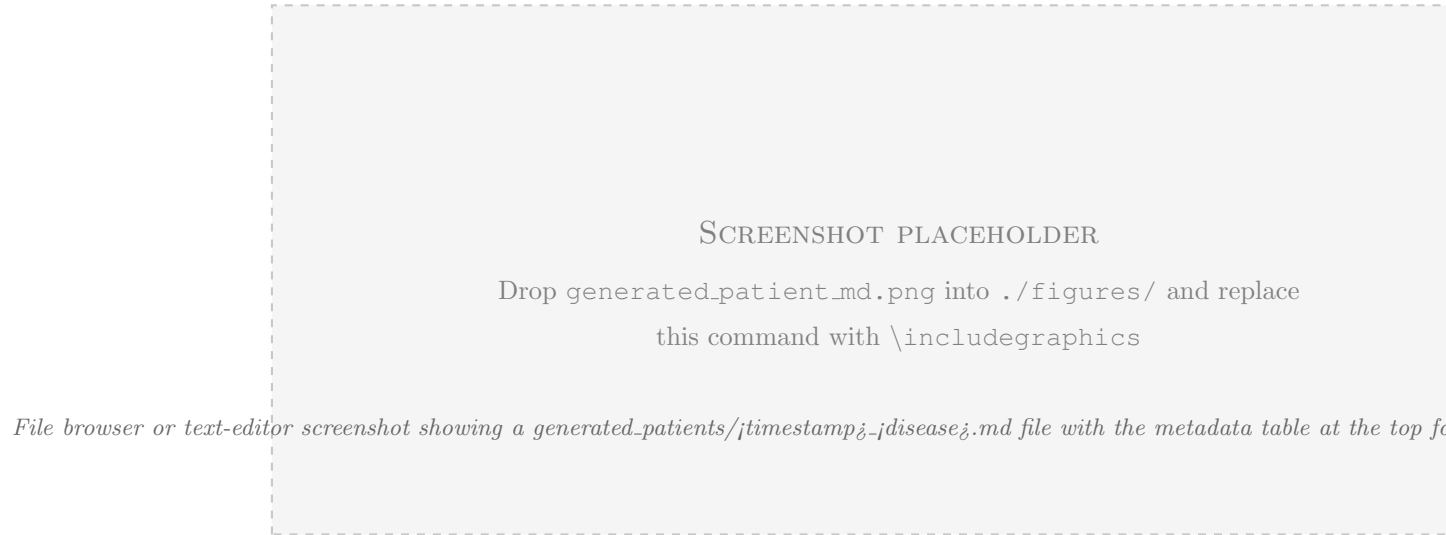


Figure 15: Persisted patient artifact: every generated case is saved as a Markdown file for cohort replay.

Figure 15 shows the on-disk artifact written by `save_patient_artifact()` after every successful Generator run and every predefined launch. Each file carries a metadata table (source, conversation ID, face, voice) plus the full vignette body so the same case can be re-launched across an entire cohort [19].

7 System Requirements

The system requirements below define the hardware and software needed to run Quner in its OpenAI-only mode (the default and lightest configuration). Full-pipeline mode with RAG enabled requires additional resources for the embedding model and Pinecone index.

7.1 Hardware Requirements (Host / Server)

- a. **Processor:** Minimum Dual-Core CPU (Intel i3 / AMD Ryzen 3 / equivalent cloud vCPU). Recommended Quad-Core for handling concurrent sessions.
- b. **Memory (RAM):** Minimum 4 GB; recommended 8 GB. The OpenAI-only path stays under 200 MB of resident memory; the RAG path adds 1–2 GB for the HuggingFace embedding model.
- c. **Storage:** 1 GB of free disk space for the application plus per-session storage for the `generated_patients/` folder (each artifact is roughly 10–20 KB).
- d. **Network:** Stable broadband (50 Mbps minimum). All LLM calls go over outbound HTTPS to the configured providers; latency is dominated by provider response time.

7.2 Software Requirements

7.2.1 Operating System (Host)

- a. Linux (Ubuntu 20.04 LTS or later, Debian 11+, or any modern Linux distribution).
- b. macOS 11 (Big Sur) or later (Apple Silicon or Intel).
- c. Windows 10 / 11 (WSL2 strongly recommended for the dashboard).

7.2.2 Runtime & Dependencies

- a. **Language Runtime:** Python 3.10 or higher (tested on 3.10 and 3.13).
- b. **Package Manager:** `pip`.
- c. **Core Python Packages (minimal install):** `openai`, `mdextractor`, `websockets`, `python-dotenv`, `nicegui<2.0`, `aiohttp`.
- d. **Optional RAG Packages:** `llama-index`, `llama-index-embeddings-huggingface`, `llama-index-vector-pinecone`, `pinecone-client`.

7.2.3 Browser (for Dashboard Use)

- a. **Recommended:** Google Chrome, Microsoft Edge, or Safari (full STT + TTS support via the Web Speech API).
- b. **Acceptable:** Firefox – TTS works; STT silently falls back to typing because Firefox does not ship `webkitSpeechRecognition`.

7.2.4 External Service Dependencies

- a. **OpenAI API key** (default deployment). Roughly \$0.02 per typical session on `gpt-4o-mini`.
- b. **Groq API key** (free tier, optional alternative). Subject to a daily token cap.
- c. **Cerebras API key** (only required for `full` pipeline mode).
- d. **Pinecone account + index** (only required for `USE_RAG=1`).

References

- [1] Rochan A, P Sowmya, and D. Anand Joseph Daniel. Large language model based document query solution using vector databases. *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*, pages 1–5, 2024.
- [2] Sofiya Azzara, Rafles, Muhammad Irwan, and Padli Nasution. Peran penting pengolahan data dalam transformasi bisnis melalui analisis. *Jurnal Rimba : Riset Ilmu manajemen Bisnis dan Akuntansi*, 2023.
- [3] Sindhu B, Prathamesh R P, Sameera M B, and Kumaraswamy S. The evolution of large language model: Models, applications and challenges. *2024 International Conference on Current Trends in Advanced Computing (ICCTAC)*, pages 1–8, 2024.
- [4] Zuhaitz Beloki, Xabier Artola, and Aitor Soroa Etxabe. A scalable architecture for data-intensive natural language processing [†]. *Natural Language Engineering*, 23:709 – 731, 2017.
- [5] Rupal Bhargava and Yashvardhan Sharma. Msats: Multilingual sentiment analysis via text summarization. *2017 7th International Conference on Cloud Computing, Data Science & Engineering - Confluence*, pages 71–76, 2017.
- [6] Holger Billhardt, Daniel Borrajo, and Victor Maojo. A context vector model for information retrieval. *J. Assoc. Inf. Sci. Technol.*, 53:236–249, 2002.
- [7] Thomas Buchholz and Claudia Linnhoff-Popien. Towards realizing global scalability in context-aware systems. In *Location- and Context-Awareness*, 2005.
- [8] Robin Chan, Katsiaryna Mirylenka, Thomas Gschwind, Christoph Miksovic, Paolo Scotton, Enrico Toniato, and Abdel Labbi. Adapting llms for structured natural language api integration. In *Conference on Empirical Methods in Natural Language Processing*, 2024.
- [9] Alexey Cheptsov, Axel Tenschert, Paul Schmidt, Birte Glimm, Mauricio Matthesius, and Thorsten Liebig. Introducing a new scalable data-as-a-service cloud platform for enriching traditional text mining techniques by integrating ontology modelling and natural language processing. In *WISE Workshops*, 2013.
- [10] Aditi Goyal, Amber Prakash Verma, Deep Kumar, and Anjali Singh. Ingenious: Text summarization and question answering. *2023 International Conference on Sustainable Computing and Smart Systems (ICSCSS)*, pages 1639–1647, 2023.
- [11] Mihajlo Grbovic, Nemanja Djuric, Vladan Radosavljevic, Fabrizio Silvestri, and Narayan L. Bhamidipati. Context- and content-aware embeddings for query rewriting in sponsored search. *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2015.
- [12] Pankaj Gupta, Ritu Tiwari, and N. Robert. Sentiment analysis and text summarization of online reviews: A survey. *2016 International Conference on Communication and Signal Processing (ICCSP)*, pages 0241–0245, 2016.
- [13] Jaemin Jeong, Daeyoung Gil, Daeho Kim, and Jaewook Jeong. Current research and future directions for off-site construction through langchain with a large language model. *Buildings*, 2024.

- [14] Zhi Jing, Yongye Su, Yikun Han, Bo Yuan, Haiyun Xu, Chunjiang Liu, Kehai Chen, and Min Zhang. When large language models meet vector databases: A survey. *ArXiv*, abs/2402.01763, 2024.
- [15] Dr Rakshitha Kiran, Samitha Khaiyum, and Aparna R Palandye. Leveraging llama3 and langchain for rapid ai application development. *Journal of Electrical Systems*, 2024.
- [16] Aku Kohli and Neetu Gupta. Big data analytics: An overview. *2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, pages 1–5, 2021.
- [17] Jan Ole Krugmann and Jochen Hartmann. Sentiment analysis in the age of generative ai. *Customer Needs and Solutions*, 11:1–19, 2024.
- [18] Chinmay Shripad Kulkarni. The evolution of large language models in natural language understanding. *Journal of Artificial Intelligence, Machine Learning and Data Science*, 2023.
- [19] Puneet Mahajan. Textual data quality at scale for high dimensionality data. *2022 International Conference on Data Science, Agents & Artificial Intelligence (ICDSAAI)*, 01:1–4, 2022.
- [20] Ali Mansourian and Rachid Oucheikh. Chatgeoai: Enabling geospatial analysis for public through natural language, with large language models. *ISPRS International Journal of Geo-Information*, 2024.
- [21] Umang Mavani, Devashri Shinde, Aditi Pednekar, and Safa Hamdare. Natural language processing based text summarization and querying model. *2020 Fourth International Conference on Inventive Systems and Control (ICISC)*, pages 806–810, 2020.
- [22] Solmaz Seyed Monir, Irene Lau, Shubing Yang, and Dongfang Zhao. Vectorsearch: Enhancing document retrieval with semantic embeddings and optimized search. *ArXiv*, abs/2409.17383, 2024.
- [23] Shantanu Pathak and Delip Rao. Adaptive system for handling variety in big text. 2018.
- [24] Beth Plale. Big data opportunities and challenges for ir, text mining and nlp. *Proceedings of the 2013 international workshop on Mining unstructured big data using natural language processing*, 2013.
- [25] Tetiana Portovaras, Nadiia Kovalenko, Anastasiia Kaplina, Natalia Kyrychenko, and Zinoviia Zaloha. Current trends and future prospects in business management analysis integration. *Multidisciplinary Reviews*, 2024.
- [26] Mengchao Ren. Advancements and applications of large language models in natural language processing: A comprehensive review. *Applied and Computational Engineering*, 2024.
- [27] Jan Rygl, Jan Pomikalek, Radim Reherek, Michael Ruzicka, Vit Novotny, and Petr Sojka. Semantic vector encoding and similarity search using fulltext search engines. In *Rep4NLP@ACL*, 2017.
- [28] Manikanta S. Content summarization and question answering system using llm. *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, 2024.
- [29] Dipankar Sarkar. Navigating the knowledge sea: Planet-scale answer retrieval using llms. *ArXiv*, abs/2402.05318, 2024.
- [30] Nazeer Shaik. The nexus of ai and vector databases: Revolutionizing nlp with llms. *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, 2024.

- [31] Zhengru Shen, Hugo van Krimpen, and Marco René Spruit. A lightweight api-based approach for building flexible clinical nlp systems. *Journal of Healthcare Engineering*, 2019, 2019.
- [32] Connor Stokes, Anoop Kumar, Frederick Choi, and Ralph M. Weischedel. Scaling nlp algorithms to meet high demand. *2015 IEEE International Conference on Big Data (Big Data)*, pages 2839–2839, 2015.
- [33] Oguzhan Topsakal and Tahir Cetin Akinci. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. *International Conference on Applied Engineering and Natural Sciences*, 2023.
- [34] Christos Trochoutsos and Yannis Sofias. The importance of data analysis in the modern era of print production. *Proceedings - The Eleventh International Symposium GRID 2022*, 2022.
- [35] P.ROHITH Varma. Natural language processing in the era of big data. *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, 2024.
- [36] Han Xu, Ruining Zhao, Jindong Wang, and Haipeng Chen. Restful-llama: Connecting user queries to restful apis. In *Conference on Empirical Methods in Natural Language Processing*, 2024.
- [37] Wenxuan Zhang, Yue Deng, Bing-Quan Liu, Sinno Jialin Pan, and Lidong Bing. Sentiment analysis in the era of large language models: A reality check. *ArXiv*, abs/2305.15005, 2023.
- [38] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Z. Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jianyun Nie, and Ji rong Wen. A survey of large language models. *ArXiv*, abs/2303.18223, 2023.